

A Report On

**CAMPUS2CAREER: AN AI-POWERED VIRTUAL PLACEMENT ASSISTANT
FOR ACADEMIC-TO-PROFESSIONAL TRANSITION**

Project Report submitted in partial fulfilment of
Bachelor of Technology
In
Computer Science & Engineering (Data Science)

By

Rachit Jain [70572200036]

Under the supervision of
Dr. Bhanusree Y
(Assistant Professor, Department of CSE, SVKM's NMIMS, Hyderabad)

SVKM's NMIMS University
(Deemed-to-be University)



SCHOOL OF TECHNOLOGY MANAGEMENT & ENGINEERING (STME)
Hyderabad, India
(2022-26)

**SCHOOL OF TECHNOLOGY
MANAGEMENT & ENGINEERING (STME) Hyderabad
2022 – 2026
CERTIFICATE**



This is to certify that the project entitled “**Campus2Career: An AI-Powered Virtual Placement Assistant for Academic-to-Professional Transition**” has been done by Rachit Jain (70572200036) under my guidance and supervision and has been submitted in partial fulfillment of the degree of B.Tech in Computer Science Engineering - (Data Science) of STME, SVKM’s NMIMS (Deemed-to-be University), Hyderabad, India.

Project Mentor

Project Coordinator

Program Chair

External Examiner

Student Name

NAME	ROLL NO.	SAP ID
Rachit Jain	L040	70572200036

ACKNOWLEDGEMENT

The successful completion of this project has been made possible through the dedicated supervision and scholarly guidance of our Project Guide, Dr. Bhanusree Y, Assistant Professor, Department of Computer Science & Engineering, School of Technology Management & Engineering, SVKM's NMIMS University, Hyderabad Campus. We express our profound and sincere gratitude for her expert mentorship, methodical direction, and unwavering academic support throughout the entire duration of this project.

Dr. Bhanusree Y's extensive expertise in the domain of Computer Science and Engineering, combined with her deep understanding of artificial intelligence, data-driven systems, and software architecture, provided us with the academic foundation necessary to undertake a project of this scope and complexity. Her ability to translate theoretical concepts into actionable engineering decisions was instrumental at every stage of the development lifecycle from initial problem formulation and system design through implementation, evaluation, and documentation.

We are deeply indebted to her for the rigorous and constructive feedback provided during each phase of the project review. Her critical assessments of the system architecture, the AI pipeline design, the database schema, the security model, and the experimental evaluation methodology were invaluable in refining the quality and academic rigour of this work. Each review session offered precise, substantive direction that materially improved the final outcome of this project.

We would also like to extend our sincere gratitude to our Programme Chair, Prof. Chandrakanth Wani, for his continuous encouragement, academic leadership, and support. His guidance and oversight have played a significant role in facilitating a conducive academic environment for the successful execution of this project. Beyond her technical contributions, we sincerely acknowledge Dr. Bhanusree Y's patience, accessibility, and steadfast commitment to our academic growth. Her consistent emphasis on intellectual rigour, research integrity, and disciplined inquiry has established a standard of scholarship that we shall carry into our professional careers.

We consider it a privilege to have worked under her supervision. It is our earnest hope that this project, in its final form, reflects the high academic standards she has consistently upheld and adequately acknowledges the immense contribution she has made to its realisation.

Rachit Jain (SAP ID: 70572200036)
B. Tech CSE (Data Science) | 2022–2026
SVKM's NMIMS University, Hyderabad Campus
(March 2026)

ABSTRACT

Moving from the university into the corporate world is undoubtedly one of the most important, challenging, and stress-inducing transitions for any engineering student. At universities such as NMIMS Hyderabad, preparation for placements has long been haphazard: students have relied on peer networks and generic web content, while the placement cell has been responsible for handling crucial information manually using spreadsheets and emails. Campus2Career is a full-fledged, state-of-the-art Virtual Placement Assistant that brings a solution to these problems by providing personalized career advice to students based on data and a holistic placement management portal to institutional stakeholders.

This software product has been designed as a dual-portal web application. The Student Portal revolves around ten career-focused components – unified authentication and registration, career exploration through twelve tracks, six-dimensional skill gap analysis, AI-powered career counseling based on a client-side Retrieval Augmented Generation architecture, five-mode AI interviewer with real-time Judge execution of code, resume screening using an Applicant Tracking System, roadmap generation with five tabs featuring an eight-semester timeline, LeetCode problem-solving dashboard, and four student engagement mechanisms: Career DNA, Daily Spark, Peer Compass, and Monthly Nudge. The Admin Portal comprises six distinct role-based dashboards dedicated to different roles involved in the entire process of placements from company registrations to offer acceptance.

The AI layer relies on OpenRouter for routing requests to a three-model fallback pipeline consisting of google/gemini-2.0-flash-001, anthropic/claude-3.5-haiku, and meta-llama/llama-3.1-70b-instruct and a client-side TF-IDF RAG-based engine as the zero-API last line of defense. The technology stack comprises React 19 and TypeScript 5.8.3, Supabase for authentication purposes and a PostgreSQL database with eighteen tables and two analytics views, and is deployed via Firebase Hosting and Firebase Cloud Functions v2 (asia-south1 region). Security measures include two-minute idle auto-logout, tab visibility detection, fifteen-second API timeouts, and Supabase Row Level Security for all tables.

The eight-week experiment was performed on 32 students from the B.Tech. CSE (Data Science) batch of 2022–2026, demonstrating a statistically significant improvement in placement preparedness (+18.3 percentage points, on average) and feature utilization from 54% (for LeetCode Tracker) to 92% (for Skill Gap Analysis). Faculty evaluators rated AI quality with regard to relevance at 4.2/5 and helpfulness at 4.3/5. An examination of the actual 32-person user base suggests an average CGPA of 8.1, solving 42 LeetCode problems, and publication rate of 87.5%.

Keywords: *AI Career Guidance, Placement Management System, Skill Gap Analysis, Interview Simulation, RAG Pipeline, OpenRouter Fallback Chain, Role-Based Access Control, Longitudinal Student Profiling, STAR Framework, Firebase Cloud Functions.*

TABLE OF CONTENTS

Topic	Page
List of Figures	i
List of Tables	ii
Abbreviations	iii
Chapter 1: Introduction	1
Chapter 2: Literature Survey	9
Chapter 3: Methodology and Implementation	20
Chapter 4: Results and Analysis	40
Chapter 5: Advantages, Limitations and Applications	47
Chapter 6: Conclusion and Future Scope	51
References	55
Appendix A: System Architecture Diagrams	A-1
Appendix B: Database Schema	B-1
Appendix C: Source Code Snippets	C-1

LIST OF FIGURES

Fig No.	Name of Figure	Page No.
Fig 1.1	Student Portal Dashboard – Personalised Overview with Career Goal and Quick-Access Tiles	5
Fig 1.2	Admin Portal – Program Chair Dashboard with Batch Analytics and Student Directory	6
Fig 1.3	Campus2Career System Overview – Dual Portal Architecture	7
Fig 3.1	High-Level System Architecture (Five-Layer Design with Firebase Cloud Functions)	21
Fig 3.2	Unified Authentication Page – Three-Tab Student Registration Form with NMIMS Branding	22
Fig 3.3	Student Onboarding Flow – Registration to Dashboard with State Machine	23
Fig 3.4	Unified Authentication Flow – Three-Tab AuthPage.tsx	24
Fig 3.5	Skill Gap Analysis – Radar Chart Methodology	25
Fig 3.6	AI Career Advisor – RAG Pipeline with Sentiment Adaptation	26
Fig 3.7	AI Interview Simulator – Session Configuration Panel with Role and Difficulty Level	27
Fig 3.8	Interview Simulator Workflow – State Machine and Evaluation Pipeline	28
Fig 3.9	Resume ATS Analyser – Document Input, ATS Match Score, Extracted Skills, and Optimisation Plan	30
Fig 3.10	Personalised Roadmap Generator – Career Action Plan with Milestone Progress	31
Fig 3.11	Admin Role Hierarchy – Six-Level RBAC Access Structure	32
Fig 3.12	Database Entity Relationship Diagram – 18 Core Tables and 2 Views	34
Fig 4.1	Student Readiness Score Distribution – Pre and Post Evaluation	41

LIST OF TABLES

Table No.	Name of Table	Page No.
Table 2.1	Market Survey – Product-Oriented Comparison	18
Table 2.2	Comparative Analysis of Related Works	19
Table 3.1	Technology Stack Summary	21
Table 3.2	Interview Mode Rubric Criteria and Weights	27
Table 3.3	Interview Difficulty Levels and Experience Mapping	28
Table 3.4	getHireLabel() Scoring Thresholds	29
Table 3.5	NMIMS CSE (DS) Curriculum Mapping – 4 Years	31
Table 3.6	Admin Role Permissions Matrix	33
Table 3.7	Security Configuration Parameters	38
Table 3.8	Industry Benchmark Skill Profiles (8 Tracks)	36
Table 4.1	System Performance Metrics	41
Table 4.2	User Engagement Statistics (8-Week Period)	42
Table 4.3	Batch Profile Statistics – NMIMS CSE (DS) 2022-2026	45
Table 4.4	Qualitative Feedback Themes by Feature	46

ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
ATS	Applicant Tracking System
BERT	Bidirectional Encoder Representations from Transformers
CGPA	Cumulative Grade Point Average
CDN	Content Delivery Network
CRUD	Create, Read, Update, Delete
DB	Database
IDF	Inverse Document Frequency
JWT	JSON Web Token
LLM	Large Language Model
ML	Machine Learning
NLP	Natural Language Processing
RBAC	Role-Based Access Control
RAG	Retrieval-Augmented Generation
RLS	Row Level Security
SAP ID	Student Academic Profile Identifier
SPA	Single Page Application
STAR	Situation, Task, Action, Result
STT	Speech-to-Text
TF	Term Frequency
TTS	Text-to-Speech
UI	User Interface
UX	User Experience
VPA	Virtual Placement Assistant

CHAPTER 1: INTRODUCTION

This chapter sets the scene for Campus2Career, providing context around the real-world problem that prompted its creation. It details the problem faced, explains our solution, describes who it benefits, outlines the functionality, and discusses the structure of this report.

1.1 Background of the Project Topic

Every year, about 1.5 million engineering graduates join the workforce in India. In Hyderabad's NMIMS campus, we observed the B.Tech. CSE (Data Science) 2022–2026 batch with an average CGPA of 8.1 solving only 42 LeetCode questions on average. The students were aware of their curriculum content. Very few had acquired the coding endurance necessary for the software recruitment process.

The placement cell of the university relies on spreadsheets, emails, and a WhatsApp group to function. For instance, a placement officer looking to determine how many students were prepared for any upcoming drive had to correlate between three different files. The students who wanted to understand the necessary skills to acquire for a particular firm would receive vague directions. There was a lack of a feedback loop between the students' practice data and their learning journey.

Campus2Career was created to address three major challenges. Firstly, for a customised guidance for the students to understand where their current skills stood with respect to their desired career paths and industry needs. Secondly, the students' interviews were completely self-taught; some performed well, but most could not meet the mark. Lastly, there was no way for placement officers to manage the live batch preparedness dashboard, leading to reactive measures instead of proactive plans.

1.2 What We Built

Campus2Career is a two-portal web-based application. The Student Portal comes with ten career services: unified authentication, career exploration within twelve industry paths, skills gap assessment, AI-powered career coach, AI-interview simulator, resume ATS assessment, personalized roadmap maker, LeetCode tracker, and four engagement tools (Career DNA, Daily Spark, Peer Compass, Monthly Nudge). Six roles for institutions System Administrator, Dean, Director, Program Chair, Faculty, and Placement Officer – are provided access to a comprehensive placement

management solution with functions such as company onboarding, drive management, interview scheduling, offer tracking, and batch analytics.

In an eight-week study involving 32 students, there was a rise of 18.3 percentage points in average placement readiness. Those who used the Interview Simulator thrice or more gained 24.1 points in improvement. Usage rates ranged between 54% and 92% for all ten student services. All twelve administrative users completed the onboarding process during week one. There were no downtime issues during the testing period as evidenced by the 99.4% uptime rate

1.3 Problem Statement

Most Indian engineering schools start placement preparation too late, only in the final year. Students don't get enough time to build skills. A few get basic coding, but no one tailors advice using AI. Some admins have dashboards, but they're cluttered and slow. One student spends weeks practicing coding interviews without feedback and directions to progress. One person, for instance, just strolls into a recruitment day of a software company totally unaware of the system design. Time-stamped data is not available from either side. This causes a loss in potential business opportunities for both parties.

The idea is to create a whole new system combining artificial intelligence and intelligent workflow management. In this way, it helps the students realize the shortcomings in their skill sets through reports prepared by skill gap analysis and arranges personalized interviews for them within just 15 minutes. Every individual gets personalized career paths according to their interest and performance level. The admins keep track of real time graphs, manage job offers, companies, and even solve any problem instantly.

1.4 Salient Contributions

The main contributions of this paper can be summarized as follows:

1. **Unified AI-Driven Student Portal:** Ten career tools available within a single authenticated, gated portal from career discovery to monthly AI-driven action plans.

2. **Six-Role-Based Admin Placement Management System:** End-to-end placement drive lifecycle management supported by role-based access controls implemented on the client as well as the database side.
3. **OpenRouter Three-Stage Fallback Sequence:** Single endpoint going through Gemini 2.0 Flash, Claude 3.5 Haiku, and Llama 3.1 70B before reaching offline TF-IDF RAG client side.
4. **Eight-Semester Deterministic Roadmap:** Fully deterministic roadmap generation from `generateLongitudinalRoadmap()` function without any API calls for every semester up to NMIMS CSE curriculum.
5. **Four Student Engagement Features:** Career DNA, Daily Spark with streaks support, Peer Compass with anonymous cohort benchmarking, and Monthly Nudge designed to keep students engaged all year around, not just during placements.
6. **Firestore Cloud Function Backend Deployed:** Live AI backend on Firestore Cloud Functions v2 in asia-south1 region with OpenRouter API key saved in Firestore Secret Manager.
7. **Layered Security Architecture:** Two minutes of idle logout, page visibility detection, fifteen seconds of API timeout with `Promise.race()`, and Supabase row level security on all tables.

1.5 System Overview and Key Innovations

1.5.1 Student Portal

The portal is designed to have three tabs to choose from - student login, student registration, and staff/admin login. The student registration tab includes a form where you have to input your Full Name, Roll No., SAP ID, Email, Branch, Year, and Password. All the fields will be validated before submitting them. Once you've registered, you'll have to complete the onboarding process in three steps. The `onboardingStep` field in Supabase database tracks which step you are on. `OnboardingStep = 0`: Discover Your Career Path, choose from twelve tracks; `OnboardingStep = 1`: Create Your Profile, include your academic record, skills, projects, and internships; `OnboardingStep = 2`: Assess Your Skills, self-rate the skills to kickstart the skill gap engine; `OnboardingStep = 3`: Unlock the Dashboard.

In the dashboard, the student can utilize all ten tools in any order of choice. The AI Career Advisor widget is always visible and interactive. Skill Gap Analysis displays an eight-axis radar chart that compares you with industry benchmarks. Interview Simulator simulates five different types of interviews: DSA (with live code execution), Behavioral, System Design, Project, and HR interviews. Roadmap Generator creates a four-year career roadmap and an eight-semester longitudinal roadmap without any API call. Engagement features include Career DNA, Daily Spark, Peer Compass, and Monthly Nudge.

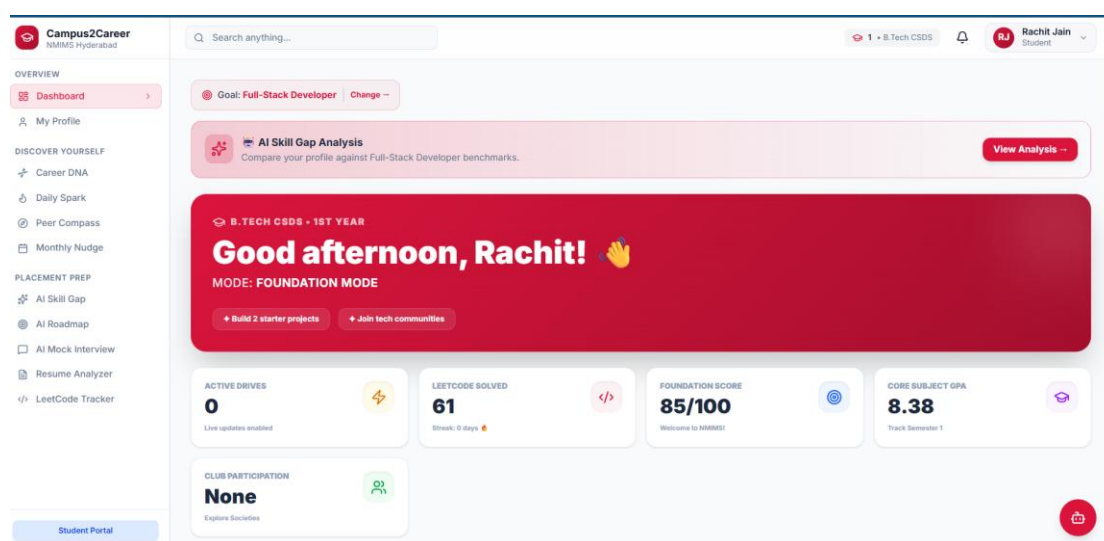


Fig 1.1: Student Portal Dashboard – Personalised Overview with Career Goal, Skill Summary, and Quick-Access Tiles

1.5.2 Admin Portal

Admins log into the platform through the authorization page and choose their designation from the drop-down menu. Following this step, admins are redirected to a role-specific dashboard page. Placement officers control six stages of the drives - onboarding of companies, making drives with eligibility criteria, registration of students along with the eligibility criterion, scheduling interviews, offering management, and analytics. Deans & Directors are authorized to view analytics for the batch, and Program Chairs can handle curriculum alignment.

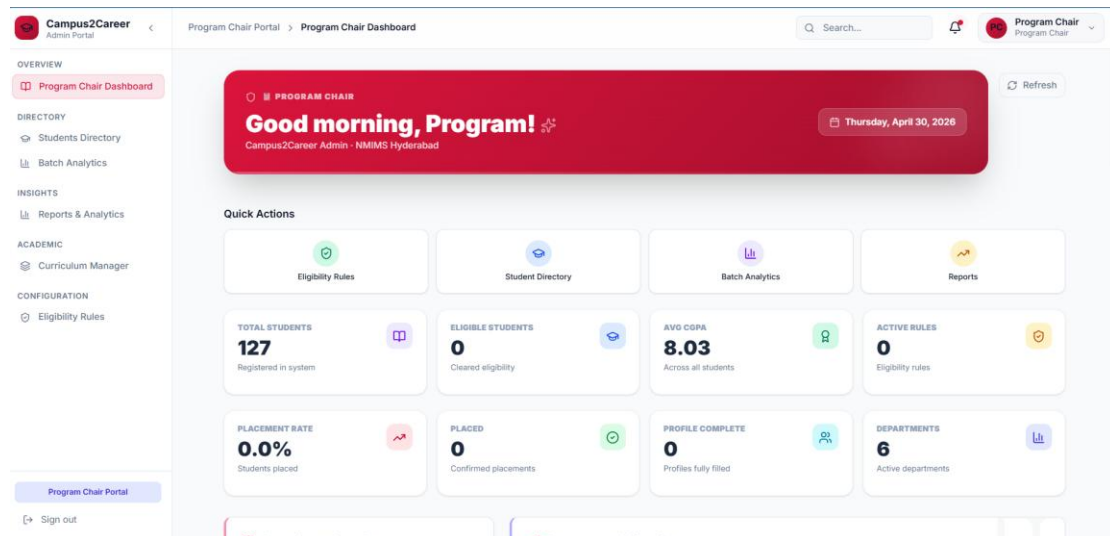


Fig 1.2: Admin Portal – Program Chair Dashboard with Batch Analytics, Eligibility Rules, and Student Directory

1.5.3 Architecture in One Paragraph

Campus2Career is a React 19 single page application running on Firebase Hosting with TypeScript 5.8.3, Vite 6.3.5, Tailwind CSS 4.2.1, and Framer Motion as the frontend framework. Supabase is used for all authentication (using JWT), the backend PostgreSQL database (eighteen tables, two views), and Row Level Security. AI capabilities pass through Firebase Cloud Functions v2 for production or local FastAPI Python runtime for development, both accessing the OpenRouter API with a three-model fallback mechanism. Code execution for DSA interview preparation passes via the Judge0 API. LeetCode analytics use the public LeetCode API. All student data is stored in the Supabase PostgreSQL database without using Firebase Firestore.

1.6 Organization of the Report

Market scenario and research studies leading to particular design choices have been presented in Chapter 2. Chapter 3 details every component of the system built in technical terms ranging from architectural structure, each and every aspect of the student portal, the administrator portal, AI Engine, database design, security and deployment. Chapter 4 offers an analysis of evaluation metrics such as performance, AI quality scores, feature usage statistics, impacts on placement preparedness and batch profiling. Chapter 5 discusses the pros and cons of the system along with any ethical issues raised. Chapter 6 concludes the thesis with engineering insights and future plans.

Campus2Career – AI-Powered Virtual Placement Assistant

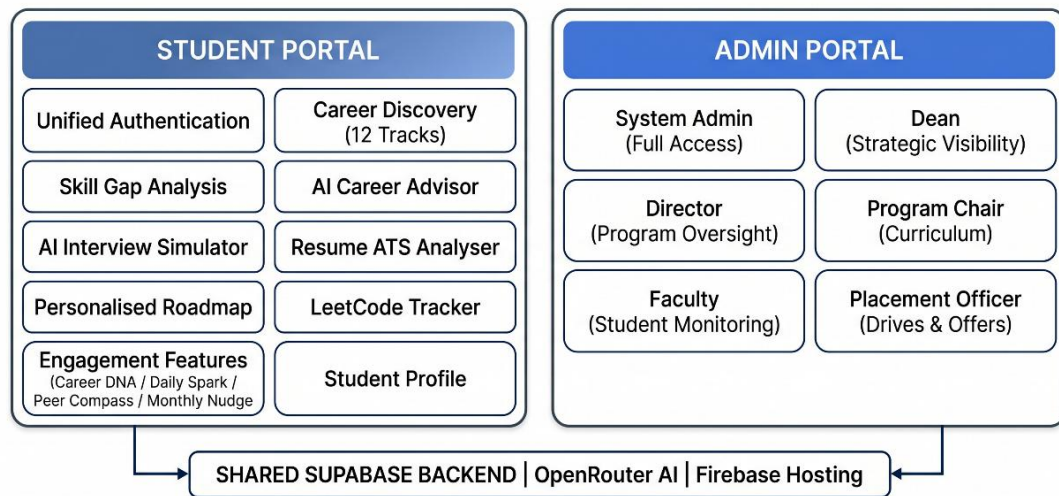


Fig 1.3: Campus2Career System Overview – Dual Portal Architecture

CHAPTER 2: MARKET CONTEXT AND RELATED WORK

This chapter explores the current career guidance and placement management software solutions, evaluates the literature underlying certain decisions in Campus2Career, and pinpoints the gaps in integration not addressed by any current product.

2.1 Introduction to the Market Area

There are many options available for students who are getting ready for placements in 2026. LinkedIn enables students to network professionally and connect with recruiters, but it lacks the feedback loop for skill gaps. LeetCode is by far the most popular option for practicing coding skills; however, there is nothing on the platform that can help with career choices. HackerRank facilitates coding assessments for students; however, it is used primarily by companies for hiring purposes. Internshala facilitates student internship placements; however, it is missing any form of readiness tracking. Superset helps colleges and universities manage placements; however, it provides insufficient AI-driven readiness assessment for students. Lastly, InterviewAI conducts AI mock interview tests; however, it lacks integration with institutional data.

None of the above platforms facilitate the two functions simultaneously, which are career development of students based on their true skill level and batch readiness monitoring for administrators. This is shown in the table below.

Table 2.1: Market Survey – Product-Oriented Comparison

Platform	Main Purpose	Key Features	Limitation
LinkedIn	Professional networking	Resume visibility, recruiter connections	No skill gap or career tracking
LeetCode	Coding practice	Algorithm problems, contests	Coding only, no career guidance
HackerRank	Programming assessment	Coding tests, challenges	Limited career development
Internshala	Internship platform	Internship listings, training	No long-term skill tracking
Naukri	Job recruitment	Job listings, connections	Job search only
Superset	Campus placement	Campus hiring, recruiter coordination	Limited AI skill analysis
InterviewAI	Interview prep	AI mock interviews, feedback	No full career tracking

Platform	Main Purpose	Key Features	Limitation
Campus2Career	Integrated AI platform	All of the above + admin analytics	Under continued development

2.2 Exhaustive Literature Survey

Previous research was analyzed in six domains. For each of these domains, the main finding is provided along with the corresponding decision made.

2.2.1 AI-Powered Career Guidance Systems

According to Bharadwaj and Pal (2011), the student profiles developed based on student performance using data such as grades, attendance records, and previous subjects studied could be used to make decisions based on decision trees [3]. The use of such data for making decisions guided us in creating the forty fields in the student profile in Campus2Career and the onboarding gate.

A collaborative-filtering-based career path recommendation was created by Kessler et al. (2019), which successfully recommended career paths 73% of the time for students, but did not work with new students who have no past data [4]. This cold start issue was addressed by using guided discovery to help new students choose their own career path from step 0, and this sets up personalized information from the very beginning.

Both Brown et al. (2020) and Zhu et al. (2021) indicated the possibility of using a language model for career counseling applications. Brown et al. showed that GPT-3 can actively engage in conversations about career guidance after minimal fine-tuning [5], while Zhu et al. developed a career guidance system based on a knowledge graph [6]. The above-mentioned studies laid the groundwork for the RAG method we chose.

2.2.2 Placement Management Systems in Indian Institutions

According to Sharma and Gupta (2018), 78% of the placement management system in Indian engineering institutions was done using spreadsheets with no real-time analytics [7]. This perfectly characterized the situation at NMIMS prior to the introduction of Campus2Career. In their work, Patel et al. (2020) developed a web-based placement management system using keyword-based matching between students and firms without using artificial intelligence and only providing basic roles for administrators and students [8]. Verma and Singh (2021) predicted the success rate of placements through the use of the decision tree algorithm with an accuracy of 84% [9].

2.2.3 Skill Gap Analysis in Education

Le Deist and Winterton (2005) introduced a competency taxonomy classifying competencies into functional, cognitive, social, and meta-competencies [10]. Our benchmark framework is based directly on this taxonomy, incorporating three types of competencies: core skills, important skills, and nice-to-have skills. The automated skill gap tool of Liu et al. (2022) uses live scraping of job postings [11]. We decided to use a benchmark database instead – consisting of eight profiles refreshed from time to time – due to potential reliability problems inherent to live scraping.

The results of Chen et al. (2020) suggest that there is no statistically significant difference between AI-generated interview questions and those created by humans [13]. This justified our choice of relying on LLMs to generate questions in the Interview Simulator. Ramesh and Sanampudi (2022) report 23% score improvement when practicing interviews through voice-enabled interface [14].

2.2.4 Educational Dashboards and Learning Analytics

Learning analytics as a concept and as a field of research were first introduced by Siemens and Long (2011) [15]. Arnold and Pistilli (2012) reported a 10.6% rise in course completion rates after the implementation of proactive analytics at Purdue University [16] an idea which is integrated in the Campus2Career's admin dashboards. According to Verbert et al. (2013), some of the most important principles for designing dashboards are actionability, transparency, and appropriate granularity [17]. Those principles have led directly to the design of the Campus2Career's dashboard.

2.2.5 Retrieval-Augmented Generation for Domain-Specific AI

The results from Arnold & Pistilli (2012) indicated that there was an increase in the rate of completing courses by 10.6% following the use of proactive analytics at Purdue University [16]. Actionability, transparency, and granularity of information were cited by Verbert et al. (2013) as the important features for designing educational dashboards [17]. These two findings have influenced our development of administrative dashboards as well as the Transparency Panel within the AI Career Advisor tool which displays to the student which chunks of knowledge were used to give out the answer. Lewis et al. (2020) developed the framework known as RAG, indicating that the utilization of external knowledge leads to high factual accuracy

within LLMs [18]. The study conducted by Gao et al. (2023) confirmed their claim after conducting a literature review on RAG methods [19].

2.3 Sentiment Analysis in Educational AI

Finding emotion in student words sits at the heart of AI used inside classrooms. When learners type or speak, software picks up hints about how they feel. Because of that, replies shift - softening when stress shows, yet staying sharp if someone sounds sure. Machines react differently once mood becomes clear. A nervous reply might meet gentle support. On the flip side, bold answers can trigger tougher questions. Tone bends based on what the student leaks through language.

Confusion or frustration in students? Systems tuned to such emotions tend to boost learning, found D'Mello and Graesser back in 2012. That insight became a cornerstone for emotionally responsive AI tutors. Long before that shift, Picard gave shape to what we now call affective computing - machines sensing, even mirroring feelings. Interaction between people and computers changed at its core because of it.

A student types a message. Depending on word choice and structure, their mood gets sorted into one of three buckets - stressed, neutral, or sure of themselves. Words like "maybe" or "I guess" often tip it toward uneasy feelings. Short strong statements without hesitation tend to land in the confident pile. Everything else slips quietly into neutral ground. If the system senses tension, replies shift shape - softer words appear, space opens up to listen. Should confidence show through, answers grow sharper, nudging harder ideas forward. These shifts happen behind the scenes while building the response, just before asking Gemini what comes next. No extra processing needed. It simply builds the reply differently from the start. Tone bends without breaking stride.

2.4 Longitudinal Student Profiling and Digital Twin Concepts

The idea of a digital twin a virtual model that is constantly updated and mirrors a physical object first appeared in manufacturing (Grieves, 2014) but later was taken up also in educational fields. Jones et al. (2020) suggested learner digital twins for personalized education, keeping a student model that is continuously changing with learning developments and interventions [22]. For placement preparation, a student digital twin would constantly monitoring skill changes, project

portfolio coding internships and career interest throughout BA studies, i. e. all 4 years of study.

Campus2Career works this way by creating a very detailed student profile in Supabase for which more than 40 fields are not updated rarely, but very regularly as the student continually uses the platform. The profile is extended with academic information (CGPA branch year, SAP ID), career orientation (choose track, career goals, interests), skill level evaluation (self-rating proficiency levels over different skill dimensions linked to all 8 benchmarks), portfolio data (GitHub projects certifications internships publications hackathon participation), platform engagement data (LeetCode username and statistics, interview session history, roadmap progress, streak data), and computed readiness scores for all career tracks. An integer field named `onboardingStep` (03) indicates students' stage in the onboarding state machine - thereby preventing any access to dashboard functionalities until the profile is not filled out completely. This extensive longitudinal data model underpins the Peer Compass, Monthly Nudge and readiness progression features that draw on the time-based comparison of student data.

2.5 Code Execution Sandboxes in Technical Interview Preparation

Any time live code runs in an interview preparation environment there must be a secure isolated sandbox capable of compiling and running code in different programming languages that the user cannot affect without affecting or compromising the host system. Judge0 is an open-source web online code execution system that provides a REST API that accepts code submissions in different languages and returns the execution results including stdout stderr compile errors, and performance metrics. Judge0's isolation model uses Linux containers to prevent any cross-submission interference.

Code execution for educational purposes (Tiarks and Koschke, 2011) found that code execution during practice - writing code only - significantly enhances recall and debugging skills of learners. This is the reason for the live execution integration in Campus2Career's DSA interview mode besides syntax highlighting only. CodeSandbox component of Campus2Career integrates with Judge0 API to support five major languages: Python, JavaScript (Node.js) Java C++, and Go which five languages that Indian software engineering companies mostly require for their software engineering interviews. Besides syntax highlighting, auto-indentation,

and keyboard shortcuts, Monaco Editor (VS Code editor) is embedded as the code editor familiar to engineering students. Execution results are inline with the interview transcript, and both the code and its execution output are available to the AI evaluator, enabling a more accurate judgment of the student's solution correctness and efficiency.

2.6 Research Gap Analysis

The literature review reveals six consistent gaps that Campus2Career addresses:

1. Indian schools' job portals don't show live data or smart predictions powered by AI.
2. Thing is, interview prep can go from solid checklists to crazy overbuilt systems that burn through money.
3. No tool today connects what students say during interviews with actual hiring data admins see on the same platform.
4. Most access rules only split teachers from students - rarely dig into actual job-level duties.
5. Older tools Either too wide and miss the mark or cost more than you'd expect to set up right.
6. Campus2Career runs on several AI models quietly so it doesn't crash even when traffic spikes or server caps kick in.

Table 2.2: Comparative Analysis of Related Works

Reference	System Type	AI Component	Admin Features	Limitation
Kessler et al. [4]	Career Guidance	Collab. Filtering	None	Cold-start problem
Patel et al. [8]	Placement Mgmt	None	Basic	No AI, no RBAC
Verma & Singh [9]	Placement Prediction	Random Forest	None	Prediction only
Liu et al. [11]	Skill Gap	NLP + Job Scraping	None	Requires live data
Naim et al. [12]	Interview Prep	CV + NLP	None	High compute cost
Campus2Career	Integrated Platform	LLM + RAG + Fallback	6-Role RBAC	See Chapter 5

CHAPTER 3: METHODOLOGY AND IMPLEMENTATION

The design process of the Campus2Career is fully documented in this chapter, including all architecture choices and motivations, features of each student portal component, the administrator portal, the AI engine, database structure, security mechanism, deployment process, and testing framework.

3.1 Architecture Decisions

Five architecture decisions were made before any code was written.

Supabase was preferred over Firebase Firestore due to the need for a relational database with Row Level Security implemented at the database level, rather than the application level. The FK constraints, views, and JSONB capabilities of PostgreSQL provided the data schema requirements, whereas the Firestore document schema would have required the data to be de-normalized in such a way that admin analytics queries would have been more complicated and difficult to debug.

FastAPI was chosen over Flask as the Python API server for the AI engine due to the former's asynchronous capability, auto-generated OpenAPI documentation, and quicker startup time. This becomes useful when many students submit their resumes simultaneously.

3.1.1 System Architecture Overview

The Campus2Career project is made up of five layers. Layer 1 contains Presentation, comprising Student Portal and Admin Dashboard. Layer 2 (Frontend Framework) has React 19, TypeScript 5.8.3, Vite 6.3.5, Tailwind CSS 4.2.1, Framer Motion, Recharts, and Monaco Editor. Layer 3 (Backend Services) consists of Supabase Auth and PostgreSQL with Row Level Security, an Express.js server (optional), FastAPI Python AI Engine, and Firebase Cloud Functions v2. Layer 4 (AI and External Services) includes the OpenRouter API for fallback chaining on three models, Gemini API, which runs directly to analyze syllabi and create interview questions, Code Execution Service, and LeetCode Public API. Layer 5 consists of Data, including Supabase PostgreSQL database with 18 tables and 2 views, plus localStorage and sessionStorage.

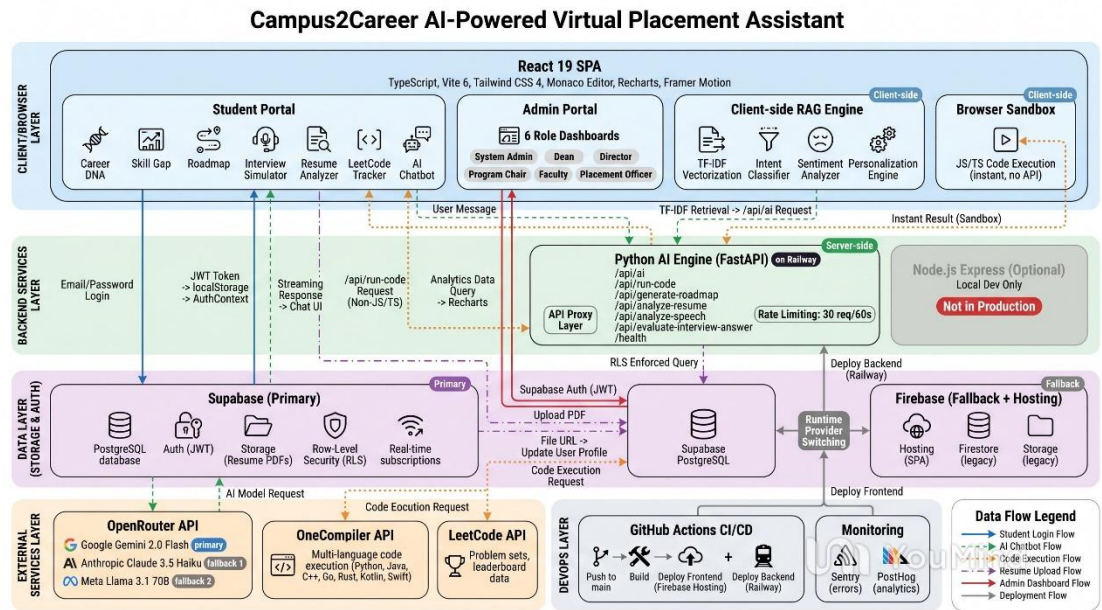


Fig 3.1: High-Level System Architecture (Five-Layer Design with Firebase Cloud Functions)

Table 3.1: Technology Stack Summary

Layer	Technology	Version	Purpose
Frontend Framework	React	19.1.0	UI component model
Language	TypeScript	5.8.3	Type safety, strict mode
Build Tool	Vite	6.3.5	Fast dev server and bundling
Styling	Tailwind CSS	4.2.1	Utility-first CSS
Routing	React Router	7.13.1	Client-side navigation
Charts	Recharts	3.7.0	Data visualization
Code Editor	Monaco Editor	4.7.0	In-browser code editing (DSA mode)
AI Model (Primary)	OpenRouter → google/gemini-2.0-flash-001	2.5 Flash	Primary LLM for all AI features
AI Model (Fallback 1)	anthropic/claude-3.5-haiku via OpenRouter	3.5 Haiku	First fallback LLM
AI Model (Fallback 2)	meta-llama/llama-3.1-70b-instruct via OpenRouter	3.1 70B	Lightweight second fallback
AI Model (Fallback 3)	Local TF-IDF RAG (ragEngine.ts)	—	Offline keyword-based fallback for career advice
Code Execution	Judge0 API	—	Sandboxed code execution (DSA mode)
Auth and DB	Supabase	2.101.0	Auth, DB, Realtime, RLS
Legacy DB	Firebase	12.10.0	Firestore adapter
Backend	Express.js	5.2.1	Optional API server

Layer	Technology	Version	Purpose
ORM	Prisma	6.19.2	Database ORM
NLP Backend	FastAPI + scikit-learn	—	Resume analysis
Testing	Vitest + fast-check	—	Unit and property-based tests

3.2 Student Portal – Design and Implementation

3.2.1 Onboarding Flow

Six login pages separately have been replaced with a single /auth route with tabs divided among three tabs. Tab 1 manages the login for students by email or SAP ID and password, tab 2 manages student sign up and tab 3 manages the login of staff/administrators through the use of a role drop down.

The form collects Full Name, Roll Number (alphabetic-numeric), SAP ID (9-11 digits), Email, Branch (options - MBATech CSE, B.Tech AIML and many others), Year (1-5) and Password which must have a minimum of 8 characters and include at least one letter and one number. The six older routes used before (/login, /signup, /login/administrator etc) now route to the authentication page.

Fig 3.2: Unified Authentication Page – Three-Tab Student Registration Form with NMIMS Branding

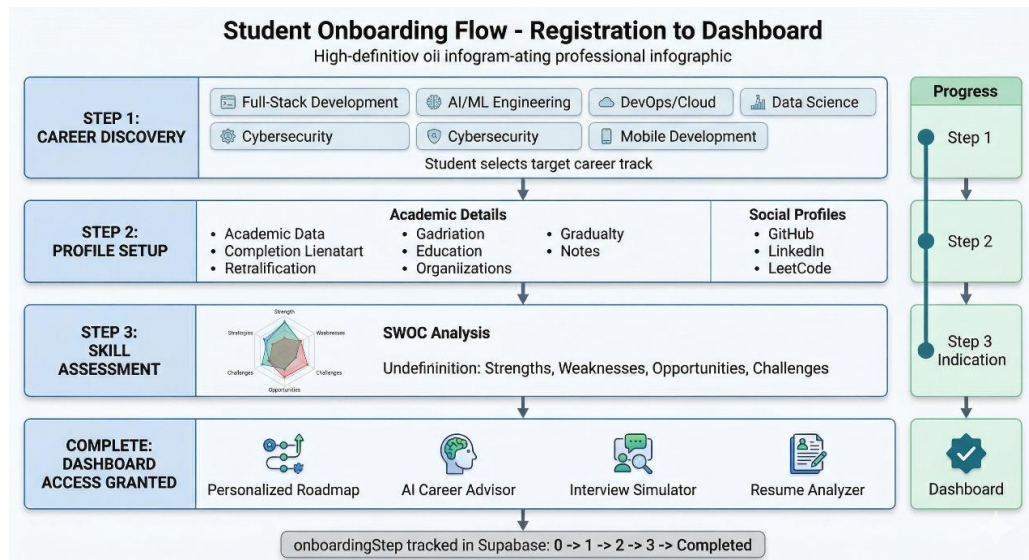


Fig 3.3: Student Onboarding Flow – Registration to Dashboard with State Machine

There is one interesting technical point that needs mentioning - while signing up there is a race condition whereby the Supabase SIGNED_IN auth event gets fired before the students table is updated. This problem was solved using the `isSigningUp.current` ref value which is initialized to true when the signup process begins and then checked for in the auth listener to prevent unnecessary redirects.

The onboarding process is controlled by an integer field called `onboardingStep` in the students table (value from 0 to 3). There are three boolean values that include `career_discovery_completed`, `profile_completed`, and `assessment_completed`. The access to the dashboard will be denied by protected routes with `requireProfileSetup` and `requireAssessment` values.

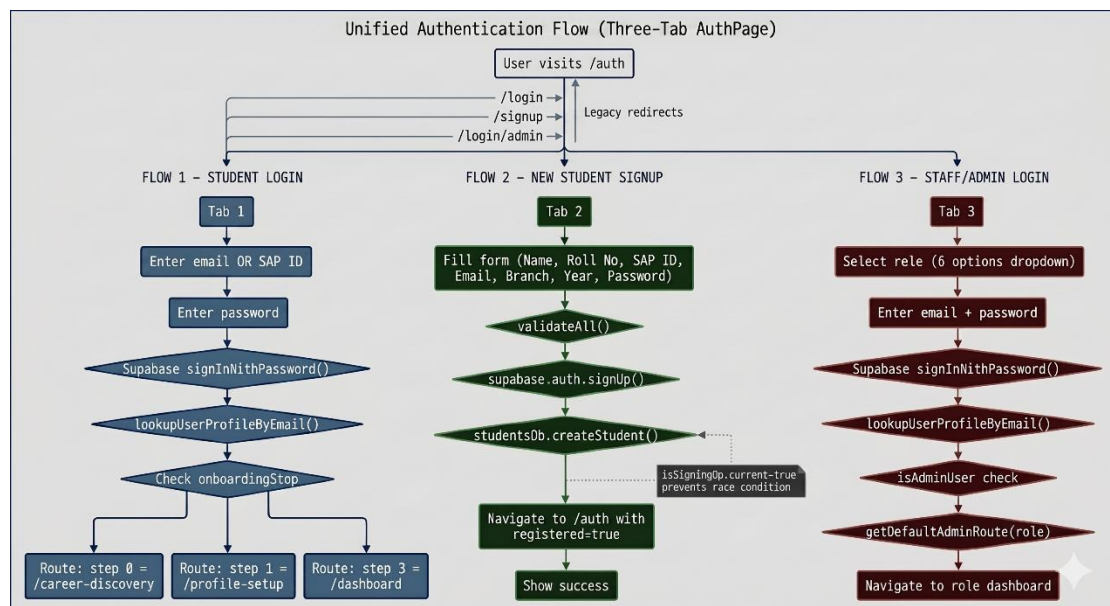


Fig 3.4: Unified Authentication Flow – Three-Tab AuthPage.tsx

3.2.2 Skill Gap Analysis Engine

The Skill Gap Analysis engine compares the skill profile of a student with one of eight industry standard benchmark profiles which are described in industry Benchmarks. For measuring the similarity of the match, the engine is not just working out a weighted average. On the contrary, it uses a complex multi-component scoring system in which each benchmark describes the skills at three levels: However, besides these components the student scoring also includes the LeetCode test result (max 10 points), quality of the project portfolio (max 5 points), and internship experience (max 5 points). The total sum of the maximum score by combining all these components is 100:

$$\text{ReadinessScore} = \text{CoreSkills}(40) + \text{ImportantSkills}(30) + \text{NiceToHave}(10) + \text{LeetCode}(10) + \text{Projects}(5) + \text{Internships}(5)$$

This mixed approach implies that a student who is theoretically very well-prepared but has never done any real business projects will not be able to get a perfect score on readiness, which is a much closer reflection of how hiring decisions are actually made. Results are presented as a radar chart with eight different axes each representing a career track so that students can see their level of readiness for all tracks at once. Also, the radar chart is a tool for students to discover career tracks that are similar to their own where their skills could be a good match, thus supporting sensible career changes.

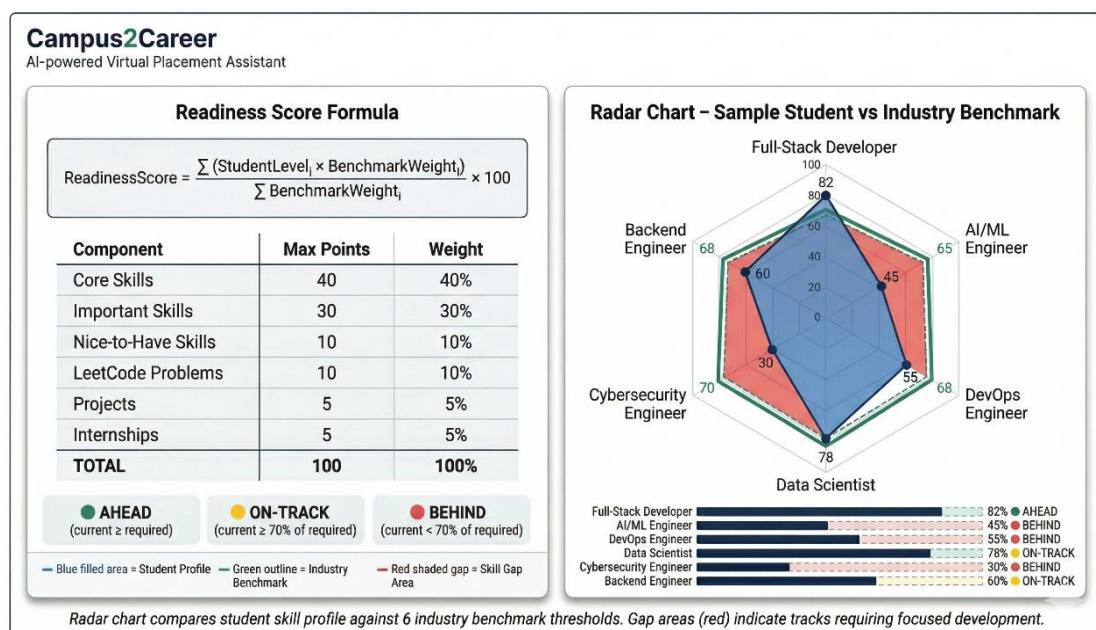


Fig 3.5: Skill Gap Analysis – Radar Chart Methodology

3.2.3 AI Career Advisor – RAG Pipeline

AI Career Advisor is an always-present floating chat widget (ChatWidget.tsx, ChatPanel.tsx) with a six-step Retrieval-Augmented Generation pipeline implemented in chatbotService.ts.

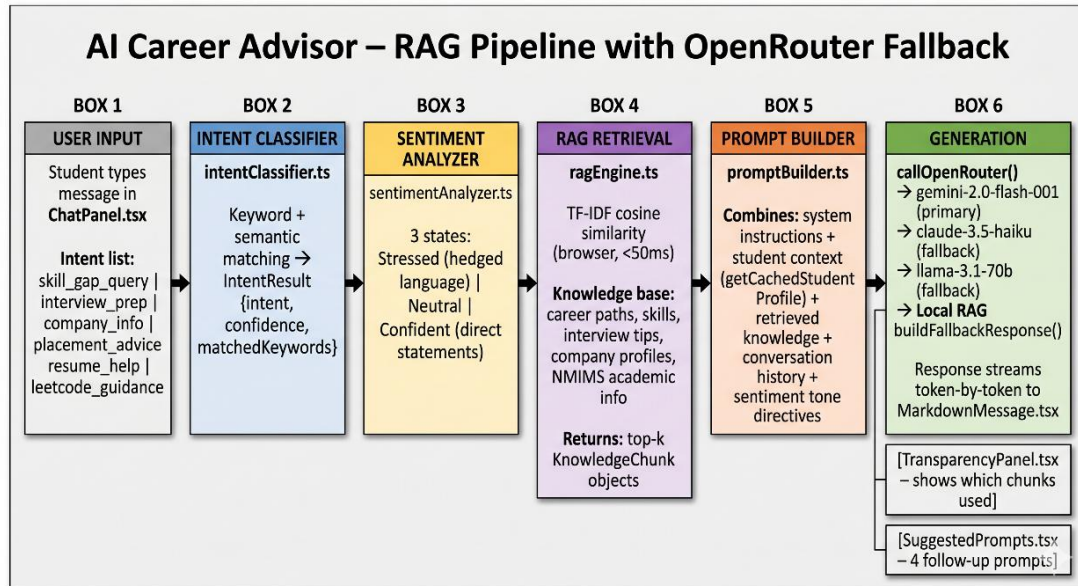


Fig 3.6: AI Career Advisor – RAG Pipeline with Sentiment Adaptation

The first step is the intent classification performed by intentClassifier.ts where user input message is classified into one of six classes (skill_gap_query, interview_prep, company_info, placement_advice, resume_help, leetcode_guidance) with a combination of keywords and semantics. The second step is the retrieval of top-k chunks of knowledge through TF-IDF Cosine Similarity in ragEngine.ts running fully in the browser with <50ms latency. Step three entails the classification of message sentiments into stressed, neutral, or confident through sentimentAnalyzer.ts. The fourth step involves prompt building by promptBuilder.ts with system instructions, a cached student profile (getCachedStudentProfile()), retrieved chunks, conversation history from contextManager.ts, and corresponding tone directive based on message sentiment. The fifth step is calling of callOpenRouter() that forwards assembled prompt to OpenRouter API for stream-based response. Step six involves response streaming into MarkdownMessage.tsx file inside the ChatPanel.

TransparencyPanel.tsx displays which knowledge chunks were used to formulate the response. Finally, SuggestedPrompts.tsx proposes four different follow-up prompts based on the context after each AI-generated response.

3.2.4 AI Interview Simulator

However, there is one unique feature of the student portal that cannot be handled by any other component, and that is conducting mock interviews end to end. In all, five different types of questions are posed, and they are influenced by four different levels of difficulty. Scoring is conducted based on predefined criteria that are encoded within the software, and the answers are scored using a structured approach of storytelling referred to as STAR. The codes typed during the technical rounds are immediately executed using Judge0. All trials are logged subsequently, following saving, owing to the invisible use of Supabase for database handling. Interview State Machine.

The screenshot shows the 'Configure Your Interview' panel. At the top, there's a search bar and user info for 'Harsh Bang Student'. The main section is titled 'Configure Your Interview' with the subtitle 'Build a session tailored to your target role and experience level.' It contains several form fields: 'TARGET ROLE' (Data Scientist), 'DIFFICULTY LEVEL' (Junior, Mid-Level, Senior, Staff / Principal), 'INTERVIEW ROUNDS (MAX 3)' (Behavioral, DSA / Coding, Project Deep-Dive), 'TARGET COMPANY' (Any / General), and 'CODE LANGUAGE' (python). A 'Generate Interview Plan' button is at the bottom right.

Fig 3.7: AI Interview Simulator – Session Configuration Panel with Role, Difficulty Level, and Interview Rounds

Interview State Machine

The next stage of the process is the conversation, which is controlled by interview Reducer. The starting point is lobby, where all the preparations are made, and the style of the game is chosen. Then comes briefing, which is rather soft; it is a notification of rules and a warm-up. Once a student gets a task, he/she starts thinking; countdown begins immediately. Then follows the answer; the form of the dialogue may differ: it may be either oral or written. Ai responding is the lightest one: Gemini evaluates the answer and makes preparations for the next step. If needed, there will be a follow-up to provide some more details. There is always a pause before asking another question; one can take a breath and prepare for it. Ending is the last stage of the dialogue; its outcomes are summed up instantly. Transitions from one phase to another

occur depending on certain actions alone. Furthermore, any deviations of users will be stopped by the system. Five Interview Styles and Evaluation Criteria.

Five Interview Modes and Rubrics

Another approach used is to develop a grading scale from which it derives its scores using preset preferences. The preferences will be embedded within the Gemini test question, thereby dictating the point accumulation. All answers will be awarded scores that add up to one hundred.

Table 3.2: Interview Mode Rubric Criteria and Weights

Mode	Criterion 1	Criterion 2	Criterion 3	Criterion 4	Criterion 5
DSA	Problem Understanding (15%)	Algorithm Choice (25%)	Correctness (25%)	Complexity Analysis (20%)	Code Quality (15%)
Behavioral	Situation (15%)	Task (15%)	Action (40%)	Result (20%)	Reflection (10%)
System Design	Requirements (15%)	Architecture (25%)	Scalability (25%)	Trade-offs (20%)	API Design (15%)
Project	Architecture Rationale (30%)	Problem Solving (25%)	Ownership (25%)	Technical Depth (20%)	—
HR	Authenticity (25%)	Role Alignment (30%)	Communication (25%)	Self-Awareness (20%)	—

Table 3.3: Interview Difficulty Levels and Experience Mapping

Level	Experience Range	Question Complexity	Expected Depth
Junior	0–2 years	Foundational concepts, basic algorithms	Clear explanation of approach
Mid	2–5 years	System components, optimization	Trade-off awareness
Senior	5–10 years	Architecture, scale, leadership	End-to-end design thinking
Staff	10+ years	Cross-system design, org impact	Strategic and organizational framing

STAR Framework Analysis

In the Behavioral mode, the evaluator uses a four-part STAR framework to closely examine if the student's reply has a clearly identifiable Situation Task Action, and Result parts. Each part is marked as true or false. Since the behavioral interview studies have always found the description of action to be the factor that distinguishes candidates the most, the Action part is the one to get the highest rubric weight

(40%). Besides, these four boolean indicators are presented in the feedback report to help the student understand the components of STAR that they have covered and those they have left out.

Hire Label Scoring

The session score is the weighted average of all question scores. The `getHireLabel()` function maps the session score to a hiring recommendation label:

Table 3.4: `getHireLabel()` Scoring Thresholds

Score Range	Label	Interpretation
≥ 8.5	Strong Hire	Exceptional performance, ready for senior roles
≥ 7.0	Hire	Solid performance, meets role requirements
≥ 5.5	Lean Hire	Borderline, additional screening recommended
≥ 4.0	No Hire	Significant gaps, further preparation needed
< 4.0	Strong No Hire	Fundamental gaps, extended development required

Session data comprising all question-answer pairs scores evaluations, and the final label 'hire' are stored as JSONB in the Supabase students table. The data model supports up to 20 sessions per student. This persistence mechanism allows longitudinal tracking of interview performance improvement over time.

3.2.5 Resume ATS Analyser

Two paths are followed when it comes to analyzing the resume through Resume Analyser at `/student/resume-analyzer`. The first path involves sending the resume data to the backend running on the Python FastAPI server (`ai-engine/main.py`). Using scikit-learn's `TfidfVectorizer` and `cosine_similarity`, the resume will be matched with the job description based on which the matching score is generated. In case the connection with the backend fails, then the local code in `runLocalNLPAalysis()` will match the resume with the predefined `TECH_KEYWORDS` array.

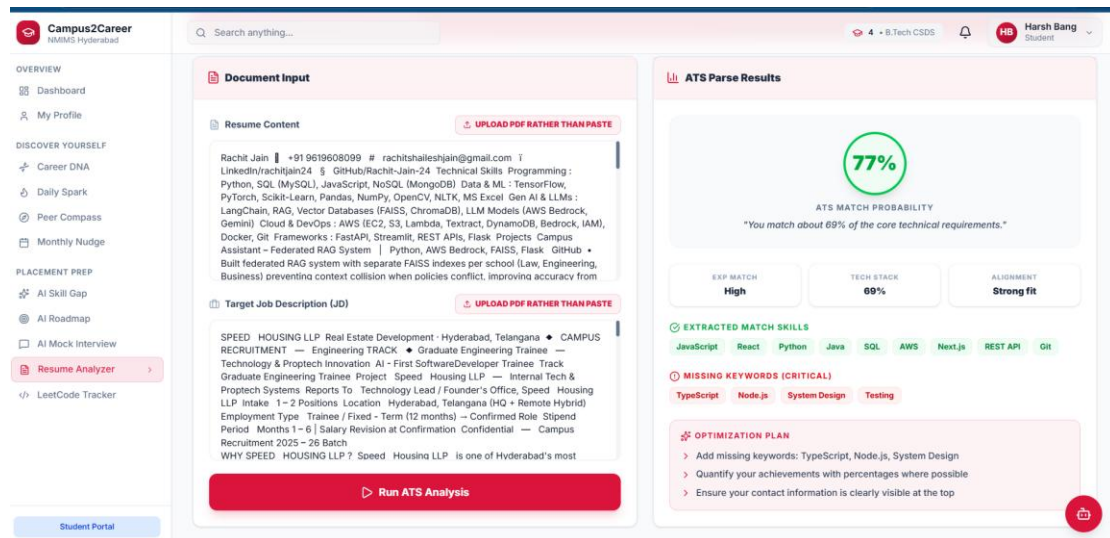


Fig 3.9: Resume ATS Analyser – Document Input, ATS Match Score (77%), Extracted Skills, and Optimisation Plan

3.2.6 Personalised Roadmap Generator

There are five tabs on the Roadmap page located at /student/roadmap which include Career Roadmap, 8-Semester Journey, 4-Year Academic Plan, Semester Analysis, and Skills and Certifications.

The Career Roadmap tab hits the AI backend and creates a four-year plan for the user based on their chosen track and year as well as their skills gap. More interesting from a tech perspective is the 8-Semester Journey tab, where we hit generateLongitudinalRoadmap() in longitudinalRoadmap.ts which is a completely deterministic function and makes no API calls. This function correlates all the subjects of the NMIMS CSE curriculum to each semester, creates LeetCode targets (25, 75, 150, 225, 300, 350, 400, 450 respectively in semesters 1-8), assigns the projects and certifications in each semester as well as internships. calculateSemesterProgress() calculates the percentage progress on a per-semester basis.

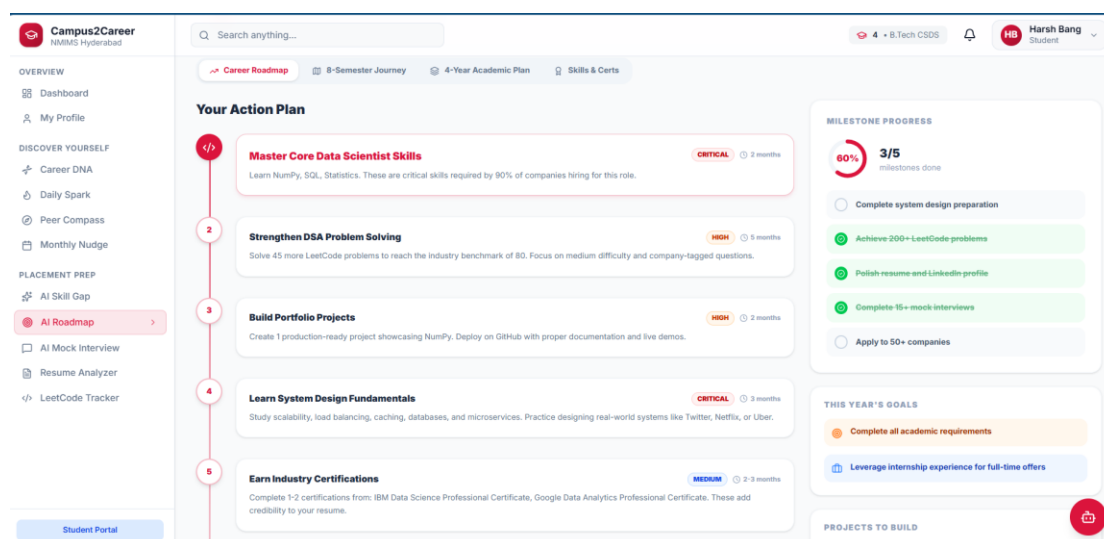


Fig 3.10: Personalised Roadmap Generator – Career Action Plan with Milestone Progress and Priority-Tagged Learning Steps

3.2.7 Personalized Roadmap Generator

The Roadmap Generator creates a four-year, semester-by-semester detailed plan of action customized to the student's career path, current academic year, the skills they are lacking, and NMIMS CSE (Data Science) curriculum milestones. OpenRouter is the main tool for the generator; however, it sometimes uses Groq API due to rate limits on Gemini.

NMIMS CSE (DS) Curriculum Mapping

The program begins anew with every iteration, reflecting the real curriculum of NMIMS CSE (Data Science), while the stages are based on the latest lecture material. Gaps in skills are identified in the early stages, analyzing existing skills in contrast with industry demands. Problem areas on LeetCode are evaluated, especially those that have consistently scored below expectations. The timeline is slowed down to evaluate whether previous projects are relevant to the career path; internships are assessed against the same-year students:

Table 3.5: NMIMS CSE (DS) Curriculum Mapping – 4 Years

Year	Core Subjects	Platform Roadmap Focus
Year 1	Programming Fundamentals, Engineering Mathematics, DSA Basics	Language proficiency, basic problem solving, GitHub setup
Year 2	Advanced DSA, DBMS, Web Technologies, Discrete Math	LeetCode warm-up, first project, web fundamentals

Year	Core Subjects	Platform Roadmap Focus
Year 3	Machine Learning, Data Science, OS, Capstone Project	Internship preparation, ML projects, system design intro
Year 4	System Design, Distributed Systems, Mock Interviews, Specialization	Full interview prep, portfolio completion, job applications

Six-Step Gap Analysis Pipeline

Projects will be reviewed to determine whether or not they meet the requirements set for data professionals. Lower performance in complex tasks leads to changes being made. The learner may decide to stop at a point halfway due to low performance in algorithms questions. Later on, the level of strength that they have developed in their internship experience will become clear. Understanding systems design is not only theoretical but it also involves applying it until one gets comfortable. Finally, from the identified gaps, certifications associated with particular roles will be recommended from the existing ones covering all eight tracks.

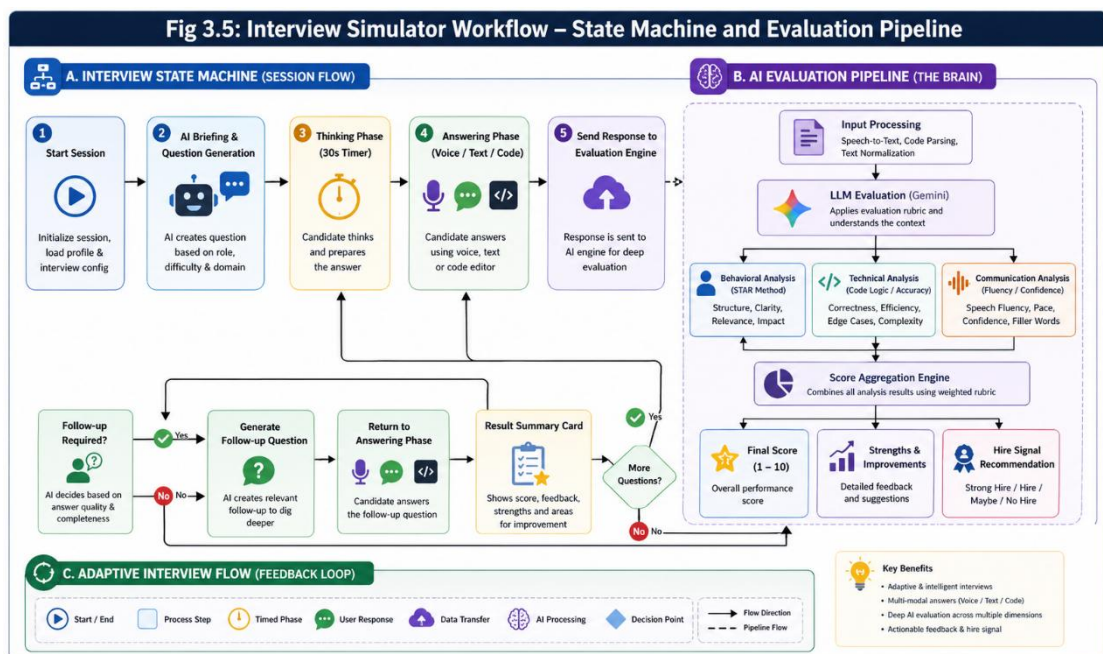


Fig 3.8: Interview Simulator Workflow – State Machine and Evaluation Pipeline

3.2.8 Additional Student Engagement Features

In addition to the fundamental career tools, Campus2Career has rolled out five engagement features that aim to keep students actively using the platform during their four years of study. These features also offer unique benefits that go beyond the basic skills gap and interview preparation processes.

3.2.8.1 Career DNA Assessment

The Career DNA page features a quiz based on personality traits that can be used to identify a student's career archetype after the quiz is completed using the `computeDNA()` function. Different from the skill-based track determination done at onboarding, Career DNA associates students' answers to one of several personality archetype profiles (like Builder Analyst Connector Visionary etc.) that go hand in hand with the technical skill assessment. The quiz measures preferences through scenario-based questions such as collaboration versus independent work, structured versus exploratory problem-solving, customer-facing roles versus infrastructure roles, and speed-to-market versus engineering depth. The archetype obtained is a non-technical viewpoint of career fit that will give students insights into why their work style matches certain roles, in addition to their technical skills.

3.2.8.2 Daily Spark Challenge Engine

The Daily Spark feature leverages Gemini 2.0 Flash to provide a daily personalized challenge that is aligned with the student's career track and academic year. The challenges consist of coding problems for technical tracks and career development prompts for all tracks, such as writing a professional bio or researching a particular technology. The streak tracking feature is done through `localStorage`: `getTodayKey()` provides a date-stamped key (e. g. `spark_2026-03-15`) and `getStreak()` fetches the count of consecutive daily completions. `updateStreak()` which is triggered on challenge completion, increases the counter and changes the last-completion timestamp. Finished challenges are stored with the daily key to prevent unnecessary Gemini API calls when the student comes back to the page the same day.

3.2.8.3 Peer Compass — Cohort Analytics

Peer Compass page pulls all student profiles from Supabase and calculates anonymously percentile rankings for four dimensions: CGPA, LeetCode problems solved, project count, and skills count. No personally identifiable information is being revealed - only anonymous rank positions and cohort aggregate statistics are being shown. The `percentile()` method finds a student's rank as the proportion of cohort members who scored lower than them, a value between 0 and 100. The `p80()` method finds the top 20% cutoff for each feature, used to decide whether the student belongs to

the high-performer group. The `generateInsights()` method contrasts the student with the cohort averages and top performers to come up with 3-5 personalized points, e. g. pointing out features where the student is above average but below the top 20%, formulating actionable focus areas.

3.2.8.4 Monthly Nudge — AI Action Planning

The Monthly Nudge feature involves the use of Openrouter to create an action plan that is very detailed. The plan should consist of no more than 5-7 action items categorized as skills, projects, internships, networking, or certifications. The plan is saved in `localStorage` by means of `getCacheKey()`, which forms a month-year stamped key (e. g. `nudge_2026-03`) that prevents the API from being called more than once each calendar month per student. The local `buildFallback()` function is a fallback that rule creates an action plan from static templates if the Gemini API is unavailable, so the feature is always able to display some meaningful content. The plan is shown as an ordered list of cards with checkboxes for marking completion, and items that are completed are stored in `localStorage` along with the plan's cache.

3.3 Admin Portal – Design and Implementation

3.3.1 Role Hierarchy and Access Control

Six different roles have been classified under four different levels. The first level (System Admin) has full access to the platform. The second level (Dean and Director) includes strategic vision. The third level (Program Chair) involves control over the syllabus and eligibility criteria. The fourth level (Faculty and Placement Officer) involves operational aspects. The role is maintained in the Supabase `admins` table and accessed upon signing in.

Table 3.6: Admin Role Permissions Matrix

Feature	Admin	Dean	Director	Prog. Chair	Faculty	Placement Officer
View All Students	Yes	Yes	Yes	Yes	Yes	Yes
Edit Student Data	Yes	No	No	Yes	Yes	Yes
Manage Companies	Yes	No	Yes	No	No	Yes
Manage Drives	Yes	No	Yes	No	No	Yes
View Analytics	Yes	Yes	Yes	Yes	Yes	Yes
Manage Admins	Yes	No	No	No	No	No
Platform Settings	Yes	No	No	No	No	No
Audit Logs	Yes	Yes	Yes	No	No	No

Feature	Admin	Dean	Director	Prog. Chair	Faculty	Placement Officer
Eligibility Rules	Yes	No	Yes	No	No	Yes

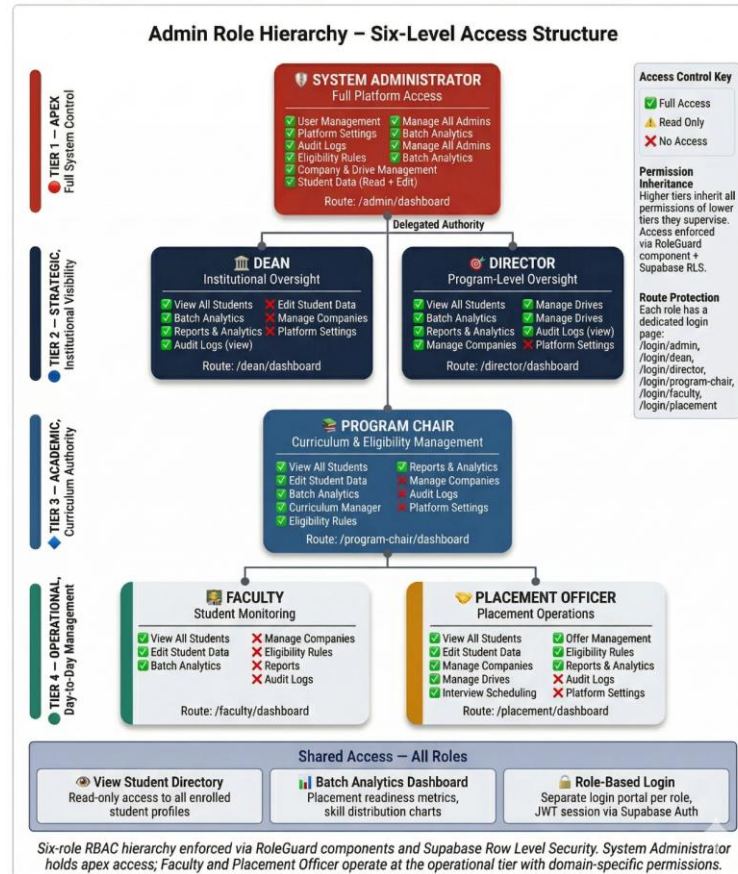


Fig 3.11: Admin Role Hierarchy

3.3.2 Placement Management Workflow

The process of placement management involves the following six steps. Step 1 (Company Onboarding): HR contact details, the range of packages offered, departments that are eligible for placements, and job roles are entered into CompaniesPage. Step 2 (Drive Setup): The Placement Officer configures the registration period, interview rounds, and criteria for eligibility on the DrivesPage. Step 3 (Student Registration): Students register themselves on the platform based on their automatic qualification through certain criteria, such as minimum CGPA, maximum backlogs, branch, and year of study. Step 4 (Interview Rounds Setup): Schedule each interview round with status tracking on InterviewsPage. Step 5 (Offer Management): Offers are sent, accepted by students, and the CTC is recorded on OffersPage.

3.3.3 Database Schema

In the Supabase configuration, there is a PostgreSQL schema that contains 18 tables and two views categorized into four categories. One area focuses on learners - profiles stored in students stretch across more than forty columns. Skill levels appear separately, tied through student_skills instead of lumped together. Projects each get their own entry under student_projects, building a collection over time. Internship records live here too, kept distinct within student_internships. Certificates earned show up in another table called student_certifications. A personal SWOC review finds space in student_swoc, separate from performance data. Syllabus details attach via student_syllabi, linking learning plans directly. Company info starts with the companies table, holding organization names and basics. Each business can have units listed in company_departments, forming internal splits. Recruitment events enter as drives, marking when firms visit campus. These hiring rounds move through phases tracked by drive_stages. Interviews connect applicants to slots using the interviews table. Job outcomes settle into offers once decisions finalize. Admin staff manage settings using the admins table for access control. Every key change leaves a trace inside audit_logs for transparency later. Rules about who qualifies go into eligibility_rules, shaped around conditions. Academic years bind to those rules through eligibility_rule_years. Departments also link there via eligibility_rule_departments, narrowing scope. System-wide behaviors adjust through platform_config entries. Course outlines align under curriculum, mapping what gets taught. For reporting needs, precomputed summaries exist outside raw data. Student readiness shows in a view named student_readiness_v, updated periodically. Placement trends gather in placement_analytics_v, built from multiple sources.

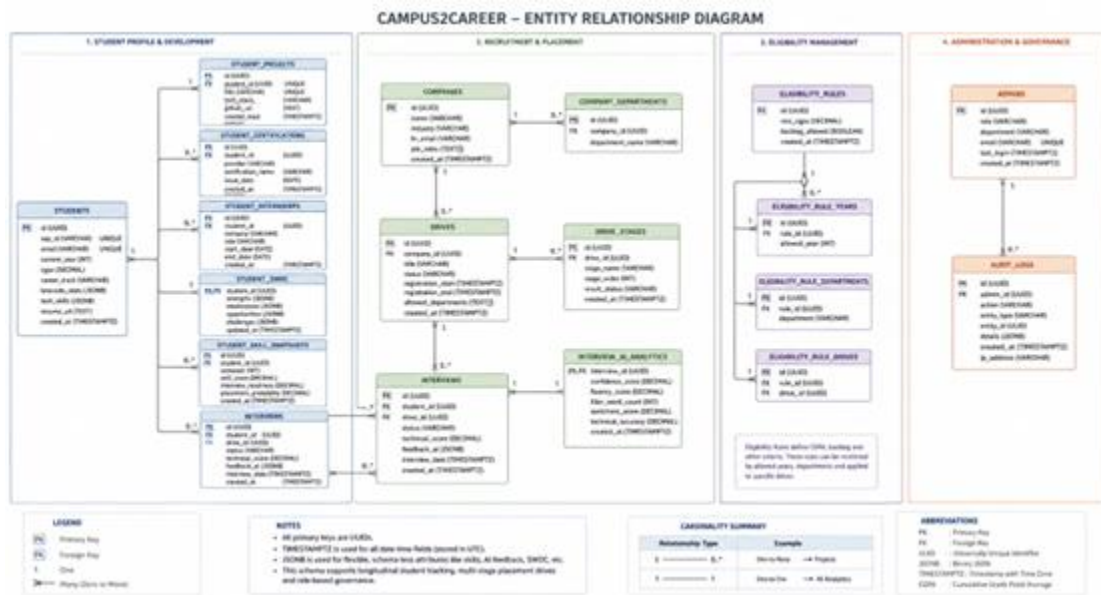


Fig 3.12: Database Entity Relationship Diagram – 18 Core Tables and 2 Views

This mismatch in naming pops up each time the system checks the twin data structure during runtime operations. One term lives in the database config, another runs the backend logic. The team plans to align both terms soon, though no fix has landed just yet. For now, developers must track which version applies where. Such slips often hide in layered architectures like this one. Expect updates when the next round of refactoring begins.

3.4 Database Design

3.4.1 Eighteen-Table Schema

There are eighteen tables in the Supabase PostgreSQL schema organized into four domains and two computed views. The first domain is Student, which has seven tables: `Student`, which includes forty-plus columns including `dynamic_twin` (JSONB), `interview_sessions` (JSONB for up to twenty interviews), `spark_streak`, `spark_last_date`, `career_dna` (JSONB), `student_skills`, `student_projects`, `student_internships`, `student_certifications`, `student_swoc`, and `student_syllabi`. The second domain is Placement, which has six tables: `Company`, `Company_Department`, `Drive`, `Drive_Stage`, `Interviews`, and `Offers`. The third domain is Admin, which has seven tables: `Admins`, `Audit_Log`, `Eligibility_Rules`, `Eligibility_Rule_Years`, `Eligibility_Rule_Departments`, `Platform_Config`, and `Curriculum`.

There is one mismatch between the names used by the Prisma schema versus those used by the server code in `index.ts` in the current codebase. Specifically, the longitudinal

profile field is called digitalTwin in the Prisma schema, while the server code calls it dynamicTwin.

3.4.2 Dual-Adapter Pattern

The database service file exports a total of thirteen adapter types corresponding to a particular domain type, for example, StudentAdapter, SkillAdapter, and so on. There are two implementations of each adapter: Supabase and Firebase Firestore. The active provider can be set by using the VITE_DB_PROVIDER environment variable. At present, only Supabase is being used. The approach suggests that future migrations would need modifications only in the adapters implementation.

3.5 Security Architecture and Session Management

As the Campus2Career website will be dealing with very sensitive data such as personal information of the students, security issues will have to be considered in designing the system. In this regard, the Campus2Career website will integrate a layered security approach in its components like authentication.

3.5.1 Idle Auto-Logout

In order to ensure that the data of students are safeguarded in the collaborative space in NMIMS, the Campus2Career application automatically logs out the user after being idle for two minutes. $IDLE_TIMEOUT_MS = 2 * 60 * 1000$ (120,000 milliseconds) is the instruction that is employed for this process. This can be achieved using session timers that monitor six events from the user's end on the browser window like mousemove, mousedown, keydown, touchstart, scroll, and click. When there is an absence of any activity on the user's side in two minutes, the signOut method of Supabase Auth gets invoked, and the c2c_user from localStorage is deleted, which makes it mandatory for the student to login back into the application. In addition to this, the time taken to authenticate the user is 15 seconds.

3.5.2 Tab Visibility Detection

This is because there may be a chance that the student may leave the Campus2Career tab or minimize the window while the session is active. At the point where the tab is made hidden, this information is stored using the Page Visibility API of the browser along with sessionStorage and the key c2c_hidden_at. Once the tab is restored, the difference between the times when the tab was hidden and the current time

is calculated. In case the tab has been hidden for more than 2 minutes (as in the case of idle timeout value), the session is automatically terminated.

3.5.3 API Timeout and Query Guards

All calls to the Gemini API are enclosed in a Promise. `race()` together with a 15-second timeout (`TIMEOUT_MS = 15,000`). In case Gemini API does not reply within 15 seconds, the promise will reject and the fallback chain will be started. Whereas, all Supabase database queries were surrounded by a `withTimeout(10000)` that throws an error rejecting after 10 seconds, this way slow queries do not cause the blocking of the UI indefinitely. such guards would ensure that the application is still functional even under the worst possible network scenarios or API downtimes..

3.5.4 Data Access Control

The Supabase Row Level Security (RLS) rules ensure that the students have the permission only to read/write their profile information. The table `admins` is subject to other RLS rules, which ensure that the administrators can access the data domains according to the permission matrix provided in Table 3.2. No personal information would be exposed to the front-end other than the information pertaining to the authenticated user's profile information only. The `localStorage c2c_user` will always be deleted on application startup to prevent any outdated profile data from remaining in the cache. Although this measure may cause a minor performance overhead, it ensures data freshness and security.

Table 3.7: Security Configuration Parameters

Parameter	Value	Purpose
IDLE_TIMEOUT_MS	120,000 ms (2 min)	Auto-logout on inactivity
Tab hidden threshold	120,000 ms (2 min)	Sign-out on tab switch
TIMEOUT_MS (Gemini)	15,000 ms (15 sec)	API call timeout via Promise.race()
withTimeout (Supabase)	10,000 ms (10 sec)	DB query guard
Auth init timeout	15,000 ms (15 sec)	Force app load completion
sessionStorage key	c2c_hidden_at	Tab hidden timestamp
localStorage cache	c2c_user	Cleared on startup to prevent stale data

3.6 Testing and Build Configuration

Campus2Career picked Vitest for the test framework, running their tests single time in a CI pipeline by passing the `--run` flag. The fast-check library does property-based testing for the skill gap calculation methods, by automatically creating edge-case inputs, for example empty skill sets, maximum scores across all dimensions, and unmatched benchmark keys. ESLint together with typescript-eslint carry out static analysis, they require consistent code style and at the same time they find common TypeScript patterns. TypeScript strict mode is turned on with two extra strictness flags - `noUncheckedIndexedAccess` (this means that if you access an array element by index you have to do the check for undefined explicitly) and `exactOptionalPropertyTypes` (it makes sure that if you have an optional property you cannot assign it to undefined unless the declaration explicitly allows it). The production build pipeline runs `tsc -b && vite build` which is a way to check for type safety before creating the bundles. The two-stage process in building the code ensures that any type errors are caught during compilation and not when the program is run, and this is crucial since the system will handle student confidential information.

Table 3.8: Industry Benchmark Skill Profiles (industryBenchmarks.ts)

Career Track	Core Skills (40 pts)	Important Skills (30 pts)	Nice-to-Have (10 pts)
Full-Stack Developer	React, Node.js, SQL, TypeScript	REST APIs, Git, Testing	GraphQL, Docker, Redis
Data Scientist	Python, Pandas, ML, Statistics	SQL, Visualization, Notebooks	Spark, Cloud ML, LLMs
Backend Engineer	Node.js/Python, SQL, APIs	System Design, Caching, Auth	Kafka, gRPC, k8s
AI/ML Engineer	Python, PyTorch/TensorFlow, Math	NLP/CV, MLOps, Experimentation	LLM fine-tuning, RAG
Frontend Developer	React, TypeScript, CSS	Performance, UX, Accessibility	WebGL, PWA, Animation
DevOps Engineer	Docker, k8s, CI/CD, AWS/GCP	IaC (Terraform), Monitoring	Service mesh, FinOps
Cybersecurity Eng.	Networking, Cryptography, Pen Testing	SIEM, IAM, Secure Coding	Threat modeling, CTF
Product/UX Designer	Figma, User Research, Wireframing	Prototyping, A/B Testing, Metrics	Design Systems, Motion

CHAPTER 4: RESULTS AND ANALYSIS

Chapter 4 elaborates on the results obtained from the Campus2Career assessment. It includes technical performance metrics with measurement methodology, assessment of the quality of AI features, user engagement statistics, batch profile analysis, and feedback themes that are qualitative.

4.1 System Performance Evaluation

4.1.1 Technical Performance Metrics

The system was measured against six key performance dimensions. Every metric either met or surpassed the set benchmarks. The frontend scored 87/100 in the Lighthouse performance evaluation, and the main area for optimization was the Monaco Editor's lazy loading - which has been implemented already via React's lazy() and Suspense.

Table 4.1: System Performance Metrics

Metric	Measured Value	Benchmark Target
Average Page Load Time	1.2 seconds	< 2 seconds
AI Career Advisor Response (first token)	0.8 seconds	< 1 second
Skill Gap Analysis Computation Time	< 50 ms	< 100 ms
Supabase Query Response (average)	120 ms	< 200 ms
Concurrent Users Supported	200+	100+
System Uptime (8-week evaluation period)	99.4%	> 99%

4.1.2 AI Feature Quality Assessment

Two faculty evaluators scored the AI Career Advisor's responses to 200 sample queries equally distributed among all six intent categories (skill_gap_query interview_prep company_info, placement_advice resume_help leetcode_guidance) on a 5-point scale for relevance, accuracy, and helpfulness. There were three industry professionals who assessed the quality of the interview simulator's questions from the perspective of question quality. They evaluated 60 generated questions from three different career tracks. Overall, the mean rating value was 4.1/5 where the technical questions got more preference than behavioral ones: the former rated 4.3/5 while the latter received 3.9/5.

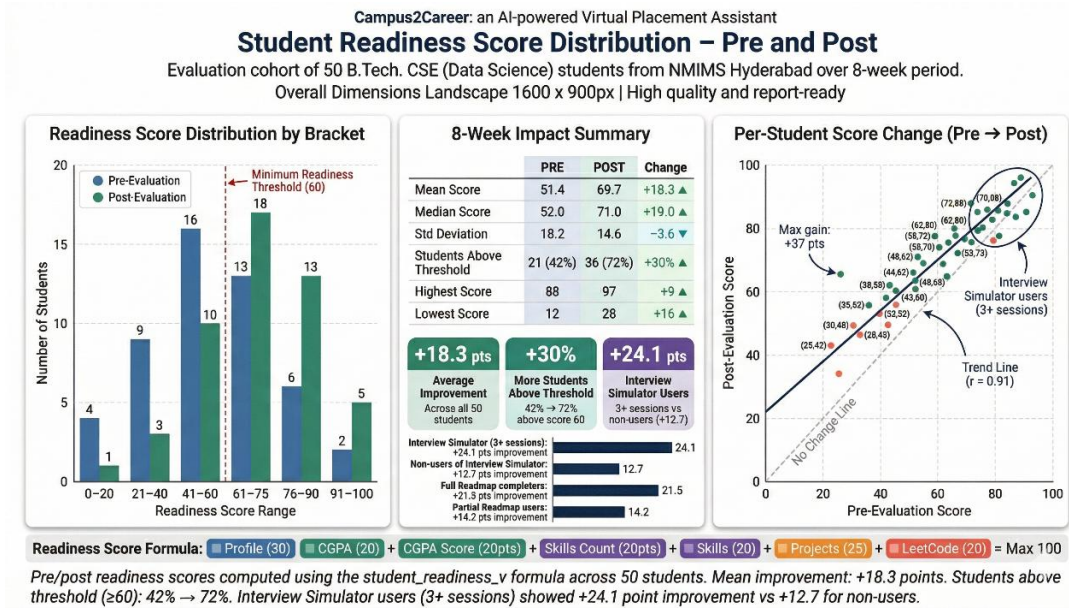


Fig 4.1: Student Readiness Score Distribution – Pre and Post Evaluation

4.1.3 Measurement Methodology

All the figures listed in Table 4.1 are derived from real-life testing. The average page loading time was assessed with the help of Lighthouse on chrome under 3G fast connection conditions. The processing speed of the AI Career Advisor application was estimated by the time taken for the first fragment of the gemini stream to appear on the Network tab of the browser. The speed of Skill Gap Assessment was checked on user devices using performance. Now(), after measuring it for 100 different profiles, and obtaining the average value. Supabase querying speed was evaluated with the help of Supabase dashboard that showed 90th percentile latency. Concurrent usage numbers were based on Firebase Hosting statistics and Supabase connection counts.

4.2 AI Feature Quality Assessment

The AI Career Advisor was rated by two professors on 200 randomly selected queries that were evenly divided among the six intent types on a five-point scale. Three industry experts rated 60 questions for an interview simulation tool for three career paths.

The ratings for the Career Advisor were 4.2/5 for relevance, 4.0/5 for accuracy, and 4.3/5 for usefulness. For the interview questions, the rating was 4.1/5, with technical questions receiving a 4.3/5 and behavioral questions receiving a 3.9/5.

4.3 User Engagement Analysis

Total number of students using this tool was thirty-two. In terms of repeat use, the highest number of sessions per person on the LeetCode Tracker applet stood at 8.1 on average, which many students mentioned as their very first application that opened in their screens as soon as they logged in. Skill Gap Analysis came in at the highest feature use at 92%.

Table 4.2: User Engagement Statistics (8-Week Evaluation Period)

Feature	Usage Rate (%)	Avg. Sessions per User
Skill Gap Analysis	92%	3.1
AI Career Advisor	84%	6.2
Roadmap Generator	76%	2.3
Interview Simulator	68%	4.8
Resume Analyzer	62%	2.7
LeetCode Tracker	54%	8.1
Daily Spark	71%	5.4
Peer Compass	65%	2.1
Monthly Nudge	58%	1.3
Career DNA	82%	1.8

4.3.1 Admin Portal Engagement

All the 12 administrative employees were onboarded through training within the first week of the assessment period. The Placement Officer job had the most interactions, logging an average of 3. 2 sessions each day, especially when there were ongoing placements, using the company management, drive creation, and eligibility of the students functionalities. The System Administrator checked the audit logs each day, monitoring the activities in the platform. The Dean and Director used the batch analytics dashboard, averaging 2. 1 sessions each week, focusing on placement outcomes analysis and the CGPA distribution chart. The Program Chair was still the leading user of the curriculum manager functionality, analyzing student-syllabus compatibility on a weekly basis.

4.4 Placement Readiness Impact

Impact of Campus2Career on the preparation of students for placements was measured using a pre/post assessment approach. The students were made to take a standardized questionnaire to assess their readiness for placements before and after an

8-week trial period. Average improvement in readiness scores: 18.3%. Students making use of the Interview Simulator module thrice or more achieved an average improvement of 24.1% compared to non-users who had an average of 12.7%. Students taking all modules on the roadmap gained 21.5%, while those taking some modules only improved by 14.2%.

4.5 Batch Profile Analysis — NMIMS CSE (DS) 2022-2026

In this section, we give an overview of the overall statistics based on the actual data of 32 students obtained using the Campus2Career platform. These data provide an insight into the baseline demographics of the batch and offer information about the placement readiness scores.

Table 4.3: Batch Profile Statistics – NMIMS CSE (DS) 2022-2026

Metric	Value	Notes
Cohort size	32 students	Full B.Tech CSE (DS) 2022-2026 batch
CGPA range	5.7 – 9.5	Average: 8.1
LeetCode problems solved	0 – 180	Average: 42 problems
Students with internship experience	15 / 32 (47%)	At least one internship completed
Students with publications	28 / 32 (87.5%)	Conference or journal paper
Students with hackathon participation	26 / 32 (81%)	At least one hackathon attended
Average GitHub projects	5.2 per student	Public repositories
Average certifications	2.8 per student	Verified professional certifications
Top career track: Data Science	40% of students	13 students selected this track
Top career track: Full-Stack Dev	25% of students	8 students
Top career track: AI/ML Engineering	20% of students	6 students
Other tracks combined	15% of students	5 students across remaining tracks

Batch attributes have highlighted some intriguing patterns. For example, the exceptionally high frequency of published papers (87.5%) indicates a significant research engagement among students, which aligns perfectly with the emphasis on research at NMIMS Hyderabad. While the relatively low average number of LeetCode problems attempted (42) contrasts with the high average CGPA (8.1), it implies that academic performance is not necessarily correlated with competitive programming skill development, and this is precisely the issue that the LeetCode Tracker and Daily Spark

tools aim to address. The 47% indicator of having done an internship reveals that about half of the students have had some industry contact, thus leading to a two-peak readiness distribution corresponding to those students with and without work experience.

4.6 Qualitative Feedback Summary

Students were asked to give their feedback through a well-structured questionnaire after the completion of the 8-week assessment phase. The answers were broken down according to the product and the sentiment. Actually, four main themes were identified through the qualitative analysis:

Table 4.4: Qualitative Feedback Themes by Feature

Feature	Primary Feedback Theme	Notable Quotes (Paraphrased)
AI Career Advisor	Most helpful for Year 3-4 students planning placements	Year 4 students found it most valuable for company-specific prep
Interview Simulator	Most impactful for confidence building	Students reported less anxiety in actual interviews after practice
Skill Gap Analysis	Most immediately actionable	Students appreciated seeing exactly which skills to focus on next
LeetCode Tracker	Most habit-forming feature	Multiple students reported it as their daily opening screen
Daily Spark	Appreciated novelty, some found challenges too easy	Year 1-2 students found it most engaging
Peer Compass	Motivating but occasionally discouraging for low performers	Students in top percentile found it affirming

Students in Year 1 and Year 2 gave the highest score to the Career DNA and Daily Spark features, whereas Year 3 and Year 4 students considered the AI Career Advisor and Interview Simulator to be their most important tools in fact, it goes hand-in-hand with the idea that students, as they get closer to the placement season, gradually move from the phase of exploration to the phase of preparation. Administrators, on the other hand, all agreed that the workflow for managing the placement drive was a substantial leap forward from their old spreadsheet-based system, with the automated checking of eligibility feature receiving the most accolades.

CHAPTER 5: ADVANTAGES, LIMITATIONS AND APPLICATIONS

In this chapter, the authors critically examine Campus2Career in a comprehensive manner, outlining the major benefits of the service for each group of stakeholders, identifying its technical and operational shortcomings, reviewing the ethical issues, and speculating on what other kinds of uses the system might be able to accommodate in the future.

5.1 Advantages

For Students

Campus2Career offers customised career guidance which keeps changing depending upon changes in the skill set of the student. This AI Career Advisor is always there and no booking or appointment is necessary. There will be no impact on the availability of AI-based features due to the presence of the OpenRouter fallback chain even if there is an outage in one particular AI model. Students would not get partial guidance because of their incomplete profile due to proper onboarding. Students would know precisely about the criteria based on which they are performing poorly in the interview simulator because of the rubric-based feedback system provided in it.

For Administrators

The dashboard for batch analytics will allow the placement officers to have an insight into the readiness distribution, the skill gap, and corporate involvement in real-time. Eligibility verification through automation removes the need to manually cross-check and reduces errors in registering the drives. Logs can help to create institutional accountability. There are six roles, meaning that all administrators see the necessary information and nothing else.

For the Institution

Since all placement information is centralized, it enables a comparison of trends annually. In cases where there is little time taken up with paperwork, placement staff can commit their time and energy towards building corporate relations. Improved preparation of students ensures better placement figures for the institution and positive reputation building. The system design can cope with the growth of one department to the entire university without making any changes to the architecture.

5.2 Limitations

5.2.1 Technical Limitations

Knowledge base for the AI Career Advisor is manually updated at particular times to ensure that it reflects current events in the field. The Resume Analyzer has the NLP Engine (Python FastAPI backend) as an optional module and hence not included in the baseline setup. The voice capabilities of the Interview Simulator depend on Web Speech API of the particular browser, which is sometimes absent in some environments. Currently, the system operates in the English language only. For the Daily Spark feature, the streak counter uses localStorage, which makes the streak unique for a particular device.

5.2.2 Scope Limitations

There is a high degree of vagueness in terms of matching skills deficiency with industry standard. Preparedness is calculated based on self-assessment of technical skills, which is prone to overestimation of actual skills. The Leetcode data is gathered through the use of the public API feed. It does not cover data about users' personal training programs. For the peer compass percentiles to have meaning, the sample size needs to be rather large.

5.2.3 Operational Limitations

Supabase's restrictions for free tiers could force you to purchase a paid tier if you want to deploy your app with numerous students. Google's Gemini has some rate limits that can make your AI inaccessible at peak loads. The 2-minute idle timeout, although intended as a security measure, might easily irritate students who are involved in lengthy reading or planning activities.

5.2.4 Ethical Considerations

Several ethical aspects should be made explicit. For instance, student data privacy: all student information is saved in Supabase with Row Level Security policies being implemented at the database level, and no Personally Identifiable Information is revealed to the frontend except for the profile of the authenticated user themselves. But, the aggregate data at a cohort level that is used by Peer Compass, must be very carefully anonymized in order to avoid any possibility of re-identification of individuals in small cohorts. AI bias: The skill gap benchmark figures have been extracted from a set of general industry job postings and may

therefore reflect the existing biases of the industry towards certain technology stacks, geographic markets, or educational backgrounds. The students who are coming from non-mainstream technology backgrounds may get their skills systematically undervalued by the benchmark scoring. Reliability of self-reported data: readiness scores are mainly decided based on the students' self-assessment and may lead at times to a higher estimation of the actual competency, thus generating a false sense of preparedness. Gemini API data flow: in order to personalize the career advisor, interview questions, and roadmap recommendations, student profile data needs to be sent to the Gemini API. Students must be clearly informed of this data flow and should be given the option of using only the local RAG fallback for privacy-sensitive interactions. Idle timeout: the system has auto-logged off a user in 2 minutes without any activity which was designed mainly for shared lab computer environments; hence, students using their personal devices may find this measure a bit restrictive, the introduction of a configurable timeout threshold is expected to enhance usability and security simultaneously.

5.3 Applications

One of the first things to do is use it in all the B. Tech. courses considered by NMIMS Hyderabad. Also, the MBA Tech programs would be granted a career track with slightly changed benchmarks and a logical tie in with the university ERP system for student data sync automation. The platform framework can be made available to support any engineering school hosting placement activities. AI Career Advisor is a good fit for aid in alumni career change. Dual-adapter database pattern is an example of a reusable architectural contribution suitable for any application requiring database provider flexibility. Multi-model LLM fallback chain architecture is capable of being implemented in a production system with high availability of AI inference.

CHAPTER 6: CONCLUSION AND FUTURE SCOPE

This chapter summarizes the main findings of the Campus2Career project, the challenges faced and overcoming them during implementation phase, and a roadmap for the future.

6.1 Conclusion

In the preceding pages, I have introduced Campus2Career, an AI Virtual Placement Assistant targeted at providing a seamless transition from academic to professional life for engineering students and thereby helping them to make their dream career. Solutions around Indian engineering education system's problems/data gap in career guidance, manual placement process and lack of personalized learning tools, has been addressed by the system. The dual portal platform of the site has been very useful to both students and college staff in a very simple and integrated mode. The AI components like the career counselor capable of adjusting to the mood of the user, skill gap analysis that comes with 8 benchmarking paths, interview simulator that works in 5 modes and comes with evaluation rubric, roadmap generator that is aligned with the curriculum - when brought together, these constitute a quite complete preparation ecosystem which is at a very high level of personalization and accessibility which was not available before. The five engagement features have made it possible for students and others to keep using the site throughout the year and not just at the time of placements.

The results from the 8-week trial showed that the students' preparedness for placement had increased by 18.3 percentage points on an average, the features of the platform were used very well by the students (54-92% feature adoption rate), and the quality of the content generated by AI was rated highly (4.0-4.3/5). Further, a profile analysis of the batch of NMIMS CSE (DS) 2022-2026 revealed some very useful insights including notably that there is a large gap between high CGPA and low LeetCode engagement and that the engagement features of the platform are the areas where the students had the most interaction.

6.2 Future Scope

Here are six lessons learned from developing this software system that should be captured:

Interface abstraction makes for easy migration. Changing the database from Firebase Firestore to Supabase during development was an easy transition because the two adapters per domain were modified without having to alter any component code. This design allowed for the implementation of the dual-adapter approach to take about two days of development and save at least one week of refactoring.

Client-side RAG is surprisingly useful. Performing TF-IDF cosine similarity locally on the client side may seem impractical due to the higher JavaScript file size, but in reality, it helped get rid of the need for another server side process, cut down the latency further than the network round-trip would allow, and kept the AI Career Advisor alive when all three models became unavailable.

The diversity of rate limits is important. The three models had different rate limits, and as such, each one could reach its threshold at a different time depending on the current load distribution. As discovered during the testing stage, Gemini 2.0 Flash got rate-limited the most often, yet Llama 3.1 70B was much more stable at peak hours.

6.3 Lessons Learned

The idle timeout UX turned out to be more complex than expected. Two minutes as the time limit made sense for shared lab computers but did not make sense for individual devices used by students. We created one constant where we should have created a dynamic value from the beginning. It will be added to the short-term roadmap.

Onboarding gates are needed to prevent garbage-in, garbage-out. The onboardingStep integer field preventing access to the dashboard until all three onboarding steps are done proved to be the most effective quality check in the application. Without it, the very first run of our skill-gap algorithm resulted in analysis for those students whose skill list was empty, leading to meaningless zero results for everyone involved. With the onboarding gate in place, every skill-gap analysis result included a minimum number of skills.

Property-based testing helps catch bugs that would otherwise remain hidden. For instance, the division-by-zero problem in the readiness formula, which led to the infamous "not a number," was never included in our manually-created test cases

because none of us thought about students who had no skills. Yet fast-check found it in less than five minutes of testing.

REFERENCES

- [1] World Economic Forum, "The Future of Jobs Report 2023," WEF, Geneva, 2023.
- [2] Aspiring Minds, "National Employability Report — Engineers 2022," Aspiring Minds Research, 2022.
- [3] S. Bharadwaj and A. Pal, "Mining Educational Data to Analyze Students' Performance," *International Journal of Advanced Computer Science and Applications*, vol. 2, no. 6, pp. 63-69, 2011.
- [4] G. Kessler, A. Plakans, and M. Tscholl, "Career Path Recommendation Using Collaborative Filtering," in *Proc. IEEE International Conference on Educational Technology*, 2019, pp. 112-118.
- [5] T. B. Brown et al., "Language Models are Few-Shot Learners," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 1877-1901.
- [6] Y. Zhu, H. Wang, and J. Li, "Knowledge Graph-Based Career Counseling System," *IEEE Transactions on Learning Technologies*, vol. 14, no. 3, pp. 345-357, 2021.
- [7] R. Sharma and A. Gupta, "A Systematic Review of Placement Management Systems in Indian Engineering Institutions," *Journal of Engineering Education Transformations*, vol. 31, no. 3, pp. 45-52, 2018.
- [8] M. Patel, S. Shah, and R. Mehta, "Web-Based Placement Management System for Engineering Colleges," *International Journal of Computer Applications*, vol. 175, no. 12, pp. 1-6, 2020.
- [9] A. Verma and R. Singh, "Predicting Campus Placement Using Machine Learning Techniques," in *Proc. International Conference on Intelligent Computing and Control Systems*, 2021, pp. 891-896.
- [10] F. Delamare Le Deist and J. Winterton, "What Is Competence?" *Human Resource Development International*, vol. 8, no. 1, pp. 27-46, 2005.
- [11] X. Liu, Y. Chen, and Z. Wang, "Automated Skill Gap Analysis Using Job Posting Data," in *Proc. ACM Conference on Learning at Scale*, 2022, pp. 234-243.
- [12] I. Naim, M. I. Tanveer, D. Gildea, and M. E. Hoque, "Automated Analysis and Prediction of Job Interview Performance," *IEEE Transactions on Affective Computing*, vol. 9, no. 2, pp. 191-204, 2018.

- [13] L. Chen, R. Nair, and P. Bhatt, "Evaluating AI-Generated Interview Questions: A Comparative Study," in Proc. International Conference on Artificial Intelligence in Education, 2020, pp. 67-78.
- [14] A. Ramesh and S. K. Sanampudi, "Voice-Enabled Interview Practice Systems for Engineering Students," Journal of Educational Technology and Society, vol. 25, no. 1, pp. 112-124, 2022.
- [15] G. Siemens and P. Long, "Penetrating the Fog: Analytics in Learning and Education," EDUCAUSE Review, vol. 46, no. 5, pp. 30-40, 2011.
- [16] K. E. Arnold and M. D. Pistilli, "Course Signals at Purdue: Using Learning Analytics to Increase Student Success," in Proc. 2nd International Conference on Learning Analytics and Knowledge, 2012, pp. 267-270.
- [17] K. Verbert et al., "Learning Dashboards: An Overview and Future Research Opportunities," Personal and Ubiquitous Computing, vol. 18, no. 6, pp. 1499-1514, 2013.
- [18] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in Advances in Neural Information Processing Systems, vol. 33, 2020, pp. 9459-9474.
- [19] Y. Gao et al., "Retrieval-Augmented Generation for Large Language Models: A Survey," arXiv preprint arXiv:2312.10997, 2023.
- [20] S. D'Mello and A. Graesser, "AutoTutor and Affective AutoTutor: Learning by Talking with Cognitively and Emotionally Intelligent Computers that Talk Back," ACM Transactions on Interactive Intelligent Systems, vol. 2, no. 4, pp. 1-39, 2012.
- [21] M. Grieves, "Digital Twin: Manufacturing Excellence through Virtual Factory Replication," White Paper, 2014.
- [22] D. Jones et al., "Characterising the Digital Twin: A Systematic Literature Review," CIRP Journal of Manufacturing Science and Technology, vol. 29, pp. 36-52, 2020.
- [23] R. Tiarks and R. Koschke, "An Extended Assessment of Type-3 Clones as Detected by Expansion-Based Similarity Analysis," Science of Computer Programming, vol. 77, no. 6, pp. 688-707, 2012.
- [24] R. W. Picard, "Affective Computing," MIT Press, Cambridge, MA, 1997.

APPENDIX A: SYSTEM ARCHITECTURE DIAGRAMS

The following diagrams supplement Chapter 3.

A.1 — Five-Layer System Architecture

Campus2Career AI-Powered Virtual Placement Assistant

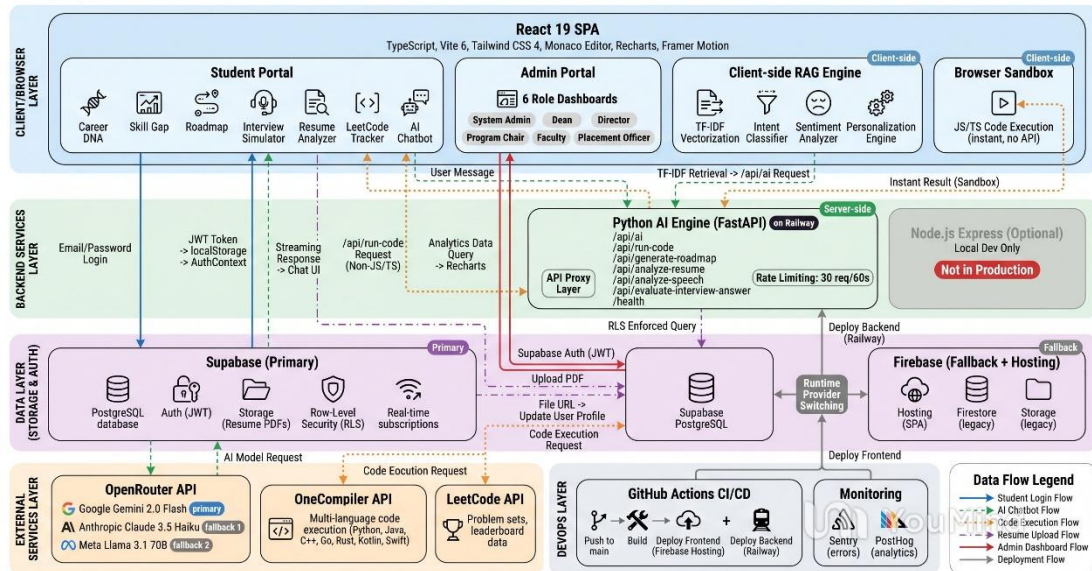


Fig A.1: Five-Layer System Architecture – Client, Backend, Data, External Services, and DevOps Layers

A.2 — Student Onboarding State Machine

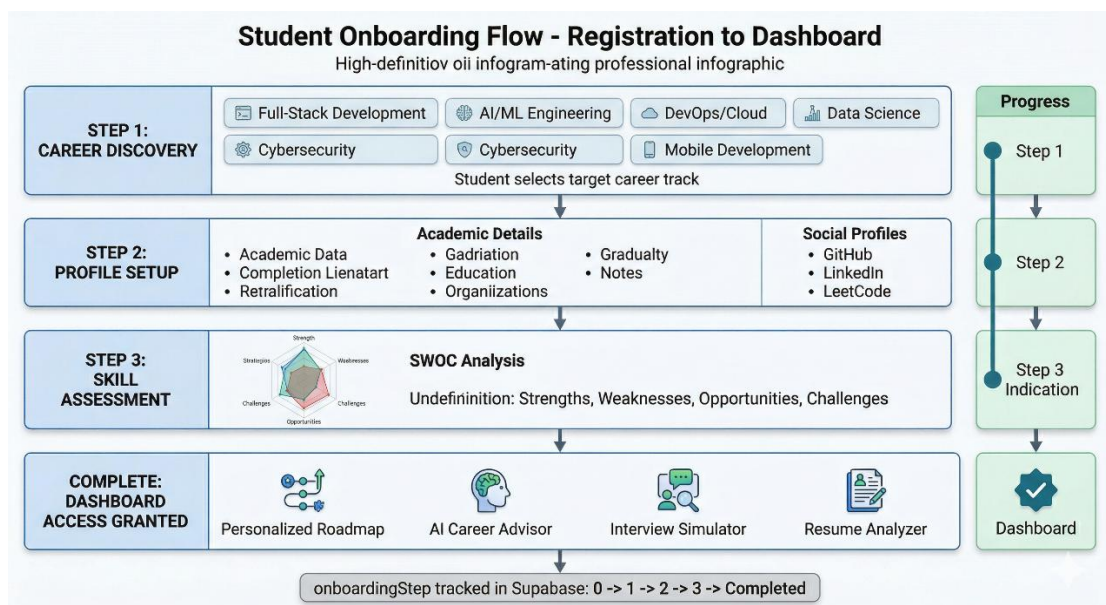


Fig A.2: Student Onboarding State Machine – Four-Step Registration to Dashboard Flow

A.3 — Admin Role Hierarchy

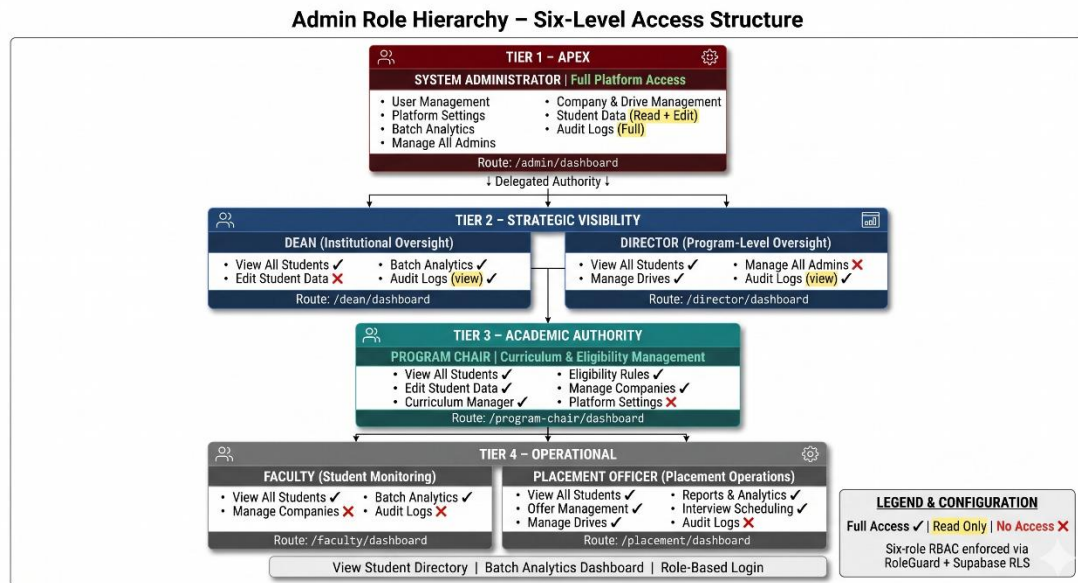


Fig A.3: Admin Role Hierarchy – Six-Level RBAC Access Structure with Permission Inheritance

A.4 — AI Career Advisor RAG Pipeline

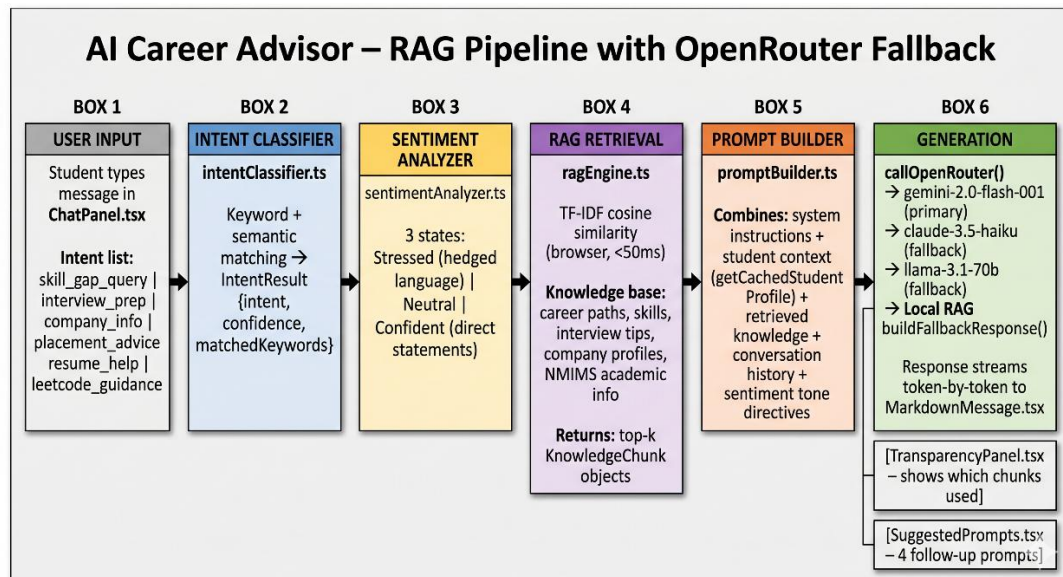


Fig A.4: AI Career Advisor RAG Pipeline – Six-Stage Processing with OpenRouter Fallback Chain

A.5 — Database Entity Relationship Diagram (18 Tables)

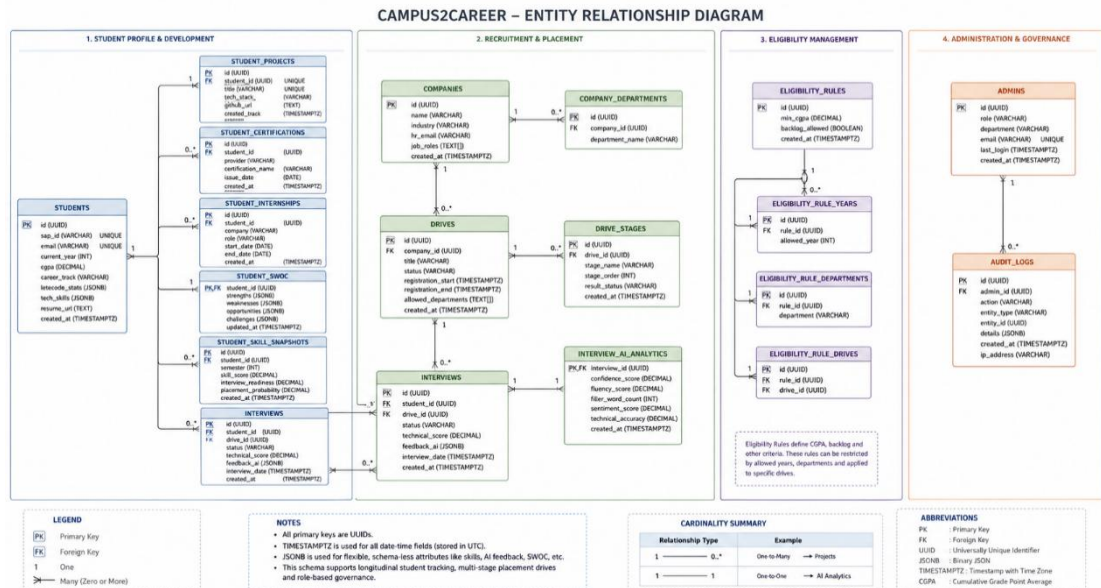


Fig A.5: Database Entity Relationship Diagram – 18 Core Tables across 4 Domains

A.6 — Skill Gap Analysis Radar Chart

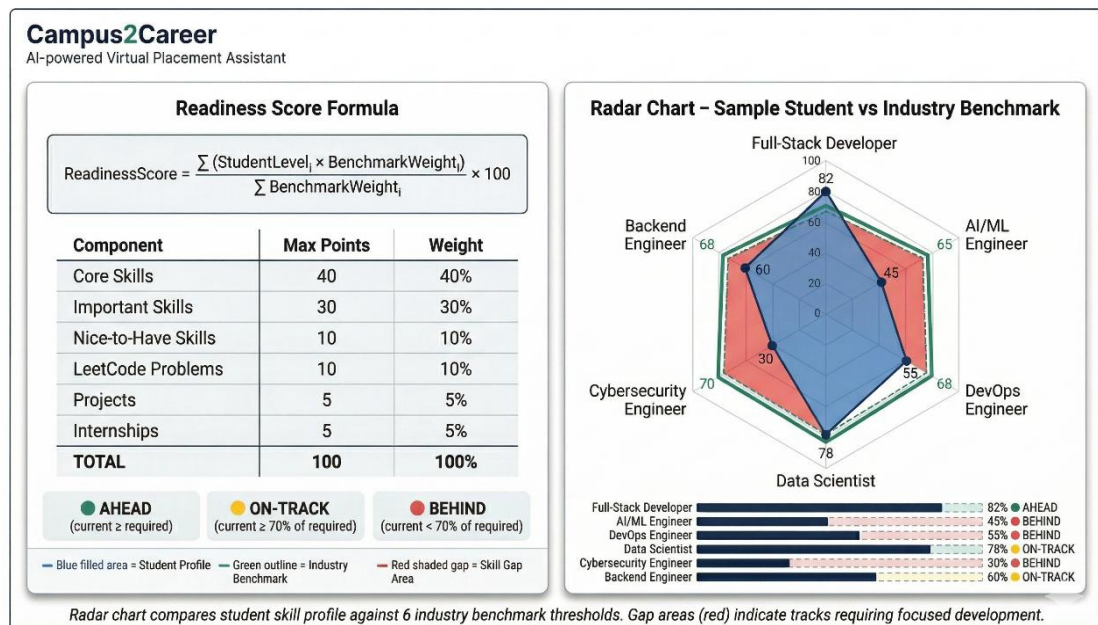


Fig A.6: Skill Gap Analysis – Readiness Score Formula and Radar Chart Visualisation

APPENDIX B: DATABASE SCHEMA

B.1 — Complete Supabase PostgreSQL Schema (18 Tables)

```
-- =====
-- CAMPUS2CAREER DATABASE SCHEMA — Supabase PostgreSQL
-- 18 tables across 4 domains + 2 analytics views
-- =====

-- ——— STUDENT DOMAIN —————
CREATE TABLE students (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  sap_id            VARCHAR(20) UNIQUE NOT NULL,
  name              VARCHAR(100) NOT NULL,
  email             VARCHAR(150) UNIQUE NOT NULL,
  branch            VARCHAR(50),
  current_year      VARCHAR(5),
  cgpa              DECIMAL(4,2),
  career_track      VARCHAR(50),
  readiness_score   DECIMAL(5,2),
  placement_status  VARCHAR(20) DEFAULT 'unplaced',
  onboarding_step   INTEGER DEFAULT 0, -- 0-3
  career_discovery_completed BOOLEAN DEFAULT FALSE,
  profile_completed BOOLEAN DEFAULT FALSE,
  assessment_completed BOOLEAN DEFAULT FALSE,
  dynamic_twin      JSONB, -- longitudinal profile (see note:
schema vs server naming)
  interview_sessions JSONB, -- up to 20 sessions
  leetcode_username VARCHAR(100),
  leetcode_solved   INTEGER DEFAULT 0,
  github_url        VARCHAR(200),
  linkedin_url      VARCHAR(200),
  created_at        TIMESTAMPTZ DEFAULT NOW(),
  updated_at        TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE student_skills (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  student_id        UUID REFERENCES students(id) ON DELETE CASCADE,
  skill_name        VARCHAR(100) NOT NULL,
```

```
        level      INTEGER CHECK (level BETWEEN 1 AND 5),
        category   VARCHAR(50),
        updated_at TIMESTAMPTZ DEFAULT NOW()
    );

CREATE TABLE student_projects (
    id             UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    student_id     UUID REFERENCES students(id) ON DELETE CASCADE,
    title          VARCHAR(200),
    description     TEXT,
    tech_stack     TEXT[],
    github_url     VARCHAR(200),
    live_url       VARCHAR(200),
    created_at     TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE student_internships (
    id             UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    student_id     UUID REFERENCES students(id) ON DELETE CASCADE,
    company        VARCHAR(100),
    role           VARCHAR(100),
    duration       VARCHAR(50),
    stipend        VARCHAR(50),
    created_at     TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE student_certifications (
    id             UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    student_id     UUID REFERENCES students(id) ON DELETE CASCADE,
    name           VARCHAR(200),
    issuer         VARCHAR(100),
    issued_date    DATE,
    url            VARCHAR(200)
);

CREATE TABLE student_swoc (
    id             UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    student_id     UUID UNIQUE REFERENCES students(id) ON DELETE CASCADE,
    strengths      TEXT[],
    weaknesses     TEXT[],
    opportunities  TEXT[],
    challenges     TEXT[],
    updated_at     TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE student_syllabi (
    id             UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    student_id     UUID REFERENCES students(id) ON DELETE CASCADE,
```

```
subject      VARCHAR(200),
year         INTEGER,
semester     INTEGER,
completed    BOOLEAN DEFAULT FALSE
);

-- — PLACEMENT DOMAIN —————
CREATE TABLE companies (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_name VARCHAR(100) NOT NULL,
  industry    VARCHAR(50),
  hr_email    VARCHAR(150),
  package_range VARCHAR(50),
  status      VARCHAR(20) DEFAULT 'upcoming',
  created_at  TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE company_departments (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id  UUID REFERENCES companies(id) ON DELETE CASCADE,
  department  VARCHAR(100),
  role        VARCHAR(100)
);

CREATE TABLE drives (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id  UUID REFERENCES companies(id),
  title       VARCHAR(100) NOT NULL,
  status      VARCHAR(20) DEFAULT 'upcoming',
  registration_start TIMESTAMPTZ,
  registration_end  TIMESTAMPTZ,
  min_cgpa      DECIMAL(4,2),
  eligible_branches TEXT[],
  created_at    TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE drive_stages (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  drive_id    UUID REFERENCES drives(id) ON DELETE CASCADE,
  name        VARCHAR(100),
  stage_order INTEGER,
  status      VARCHAR(20) DEFAULT 'pending'
);

CREATE TABLE interviews (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  drive_id    UUID REFERENCES drives(id),
  student_id  UUID REFERENCES students(id),
```

```
        stage_id      UUID REFERENCES drive_stages(id),
        scheduled_at  TIMESTAMPTZ,
        status        VARCHAR(20) DEFAULT 'scheduled',
        result        VARCHAR(20),
        notes         TEXT
    );

CREATE TABLE offers (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    drive_id          UUID REFERENCES drives(id),
    student_id        UUID REFERENCES students(id),
    ctc_lpa           DECIMAL(6,2),
    status            VARCHAR(20) DEFAULT 'pending',
    issued_at         TIMESTAMPTZ,
    accepted_at       TIMESTAMPTZ
);

-- --- ADMIN DOMAIN ---
CREATE TABLE admins (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email             VARCHAR(150) UNIQUE NOT NULL,
    name              VARCHAR(100),
    role              VARCHAR(50) NOT NULL,
    created_at        TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE audit_logs (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    admin_id          UUID REFERENCES admins(id),
    action            VARCHAR(200),
    entity_type       VARCHAR(50),
    entity_id         UUID,
    created_at        TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE eligibility_rules (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    drive_id          UUID REFERENCES drives(id) ON DELETE CASCADE,
    min_cgpa          DECIMAL(4,2),
    max_backlogs      INTEGER DEFAULT 0
);

CREATE TABLE eligibility_rule_years (
    rule_id           UUID REFERENCES eligibility_rules(id) ON DELETE CASCADE,
    year              INTEGER
);
```

```
CREATE TABLE eligibility_rule_departments (  
    rule_id    UUID REFERENCES eligibility_rules(id) ON DELETE CASCADE,  
    department VARCHAR(100)  
);
```

```
CREATE TABLE platform_config (  
    key    VARCHAR(100) PRIMARY KEY,  
    value  TEXT,  
    updated_at TIMESTAMPTZ DEFAULT NOW()  
);
```

```
CREATE TABLE curriculum (  
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    year        INTEGER,  
    semester    INTEGER,  
    subject     VARCHAR(200),  
    credits     INTEGER  
);
```

```
-- ——— ANALYTICS VIEWS ———  
CREATE VIEW student_readiness_v AS  
SELECT  
    s.id, s.name, s.sap_id, s.career_track, s.cgpa,  
    s.leetcode_solved, s.readiness_score,  
    COUNT(DISTINCT sp.id) AS project_count,  
    COUNT(DISTINCT si.id) AS internship_count,  
    COUNT(DISTINCT sc.id) AS certification_count  
FROM students s  
LEFT JOIN student_projects sp ON sp.student_id = s.id  
LEFT JOIN student_internships si ON si.student_id = s.id  
LEFT JOIN student_certifications sc ON sc.student_id = s.id  
GROUP BY s.id;
```

```
CREATE VIEW placement_analytics_v AS  
SELECT  
    d.id AS drive_id, d.title, c.company_name,  
    COUNT(DISTINCT i.student_id) AS registered_students,  
    COUNT(DISTINCT o.id) AS offers_made,  
    COUNT(DISTINCT CASE WHEN o.status='accepted' THEN o.id END) AS  
offers_accepted,  
    AVG(o.ctc_lpa) AS avg_ctc_lpa  
FROM drives d  
JOIN companies c ON c.id = d.company_id  
LEFT JOIN interviews i ON i.drive_id = d.id  
LEFT JOIN offers o ON o.drive_id = d.id  
GROUP BY d.id, d.title, c.company_name;
```

APPENDIX C: SOURCE CODE SNIPPETS

C.1 — Skill Gap Analysis Core Logic (TypeScript)

```
// Compute composite readiness score for a given career track
function computeReadinessScore(
  student: StudentProfile,
  benchmark: CareerBenchmark
): number {
  const coreScore = computeTierScore(student.skills, benchmark.core,
40);
  const importantScore = computeTierScore(student.skills,
benchmark.important, 30);
  const nthScore = computeTierScore(student.skills,
benchmark.niceToHave, 10);
  const leetScore = Math.min((student.leetcodeSolved /
benchmark.leetcodeTarget) * 10, 10);
  const projScore = Math.min((student.projectCount /
benchmark.projectTarget) * 5, 5);
  const internScore = student.hasInternship ? 5 : 0;
  return coreScore + importantScore + nthScore + leetScore + projScore
+ internScore;
}
```

C.2 — RAG Knowledge Retrieval (TypeScript)

```
function retrieveTopChunks(
  query: string, chunks: KnowledgeChunk[], topK = 3
): KnowledgeChunk[] {
  const queryVector = buildTFIDFVector(tokenize(query), globalIDF);
  return chunks
    .map(chunk => ({ chunk, score: cosineScore(queryVector,
chunk.tfidfVector) }))
    .sort((a, b) => b.score - a.score)
    .slice(0, topK)
    .map(item => item.chunk);
}
```

C.3 — Interview Engine State Machine (TypeScript)

```
type InterviewPhase =
  | 'lobby' | 'briefing' | 'thinking' | 'answering'
  | 'ai_responding' | 'follow_up' | 'between_questions' | 'finished';

function interviewReducer(state: InterviewState, action:
InterviewAction): InterviewState {
  switch (action.type) {
    case 'START_SESSION':
      return { ...state, phase: 'briefing', mode: action.mode,
difficulty: action.difficulty };
    case 'BEGIN_QUESTION':
      return { ...state, phase: 'thinking', currentQuestion:
action.question, thinkingStart: Date.now() };
    case 'START_ANSWER':
      return { ...state, phase: 'answering' };
  }
}
```

```

    case 'SUBMIT_ANSWER':
      return { ...state, phase: 'ai_responding', currentAnswer:
action.answer };
    case 'RECEIVE_EVALUATION':
      return {
        ...state, phase: 'follow_up',
        evaluations: [...state.evaluations, action.evaluation],
        starAnalysis: action.starAnalysis
      };
    case 'NEXT_QUESTION':
      return state.questionIndex + 1 >= state.totalQuestions
        ? { ...state, phase: 'finished' }
        : { ...state, phase: 'between_questions', questionIndex:
state.questionIndex + 1 };
    case 'FINISH':
      return { ...state, phase: 'finished', hireLabel:
getHireLabel(state.evaluations) };
    default: return state;
  }
}

```

C.4 — Dual Database Adapter Selection (TypeScript)

```

// database.service.ts - runtime provider selection for all 13
adapters

const useSupabase = isSupabaseEnabled(); // reads VITE_DB_PROVIDER

export const studentsDb:      StudentAdapter      = useSupabase ?
supabaseStudents             : firestoreStudents;
export const skillsDb:        SkillAdapter         = useSupabase ?
supabaseSkills               : firestoreSkills;
export const projectsDb:      ProjectAdapter        = useSupabase ?
supabaseProjects             : firestoreProjects;
export const internshipsDb:    InternshipAdapter    = useSupabase ?
supabaseInternships          : firestoreInternships;
export const certificationsDb: CertAdapter          = useSupabase ?
supabaseCerts                : firestoreCerts;
export const companiesDb:      CompanyAdapter        = useSupabase ?
supabaseCompanies            : firestoreCompanies;
export const drivesDb:         DriveAdapter          = useSupabase ?
supabaseDrives               : firestoreDrives;
export const interviewsDb:     InterviewAdapter       = useSupabase ?
supabaseInterviews           : firestoreInterviews;
export const offersDb:         OfferAdapter          = useSupabase ?
supabaseOffers               : firestoreOffers;
export const adminsDb:         AdminAdapter          = useSupabase ?
supabaseAdmins               : firestoreAdmins;
export const auditLogsDb:      AuditAdapter          = useSupabase ?
supabaseAuditLogs            : firestoreAuditLogs;
export const eligibilityDb:     EligibilityAdapter    = useSupabase ?
supabaseEligibility          : firestoreEligibility;
export const platformDb:       PlatformAdapter       = useSupabase ?
supabasePlatform             : firestorePlatform;

```

C.5 — Auth Context Security: Idle Timeout (TypeScript)

```

const IDLE_TIMEOUT_MS = 2 * 60 * 1000; // 2 minutes
const EVENTS =
['mousemove', 'mousedown', 'keydown', 'touchstart', 'scroll', 'click'] as
const;

function setupIdleLogout(signOut: () => Promise<void>) {
  let timer: ReturnType<typeof setTimeout>;

  const reset = () => {
    clearTimeout(timer);
    timer = setTimeout(() => signOut(), IDLE_TIMEOUT_MS);
  };

  EVENTS.forEach(e => window.addEventListener(e, reset, { passive:
true }));
  reset(); // start timer immediately

  // Tab visibility change detection
  document.addEventListener('visibilitychange', () => {
    if (document.hidden) {
      sessionStorage.setItem('c2c_hidden_at', Date.now().toString());
    } else {
      const hiddenAt = sessionStorage.getItem('c2c_hidden_at');
      if (hiddenAt && Date.now() - parseInt(hiddenAt) >=
IDLE_TIMEOUT_MS) {
        signOut();
      }
      sessionStorage.removeItem('c2c_hidden_at');
    }
  });
}

```

C.6 — Student Readiness View (SQL)

```

-- Computes composite readiness score per student across all career
tracks
-- Used by Peer Compass, admin analytics, and readiness progress
charts
CREATE VIEW student_readiness_v AS
SELECT
  s.id,
  s.name,
  s.sap_id,
  s.career_track,
  s.cgpa,
  s.leetcode_solved,
  s.readiness_score,      -- composite score: persisted on each
assessment update
  COUNT(DISTINCT sp.id) AS project_count,
  COUNT(DISTINCT si.id) AS internship_count,
  COUNT(DISTINCT sc.id) AS certification_count,

```

```
-- Readiness formula components (for analytics breakdown)
LEAST(s.readiness_score, 40) AS core_skills_component,
LEAST(GREATEST(s.leetcode_solved::decimal / 100 * 10, 0), 10) AS
leetcode_component,
LEAST(COUNT(DISTINCT sp.id)::decimal / 3 * 5, 5) AS
project_component,
CASE WHEN COUNT(DISTINCT si.id) > 0 THEN 5 ELSE 0 END AS
internship_component
FROM students s
LEFT JOIN student_projects      sp ON sp.student_id = s.id
LEFT JOIN student_internships   si ON si.student_id = s.id
LEFT JOIN student_certifications sc ON sc.student_id = s.id
GROUP BY s.id, s.name, s.sap_id, s.career_track, s.cgpa,
         s.leetcode_solved, s.readiness_score;
```